Introduction to deep learning

INTRODUCTION TO DEEP LEARNING IN PYTHON

Dan Becker

Data Scientist and contributor to Keras and TensorFlow libraries



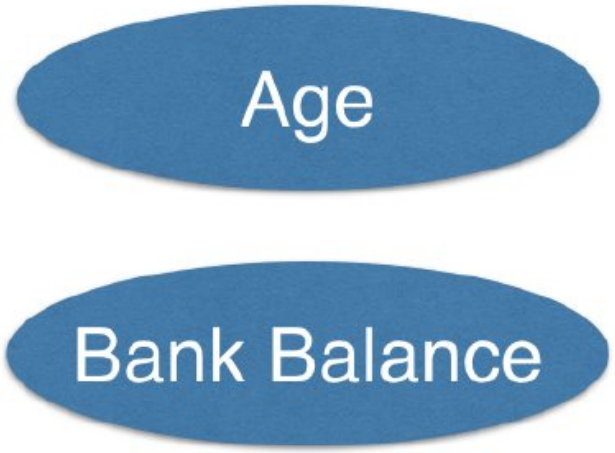
Imagine you work for a bank

- You need to predict how many transactions each customer will make next year

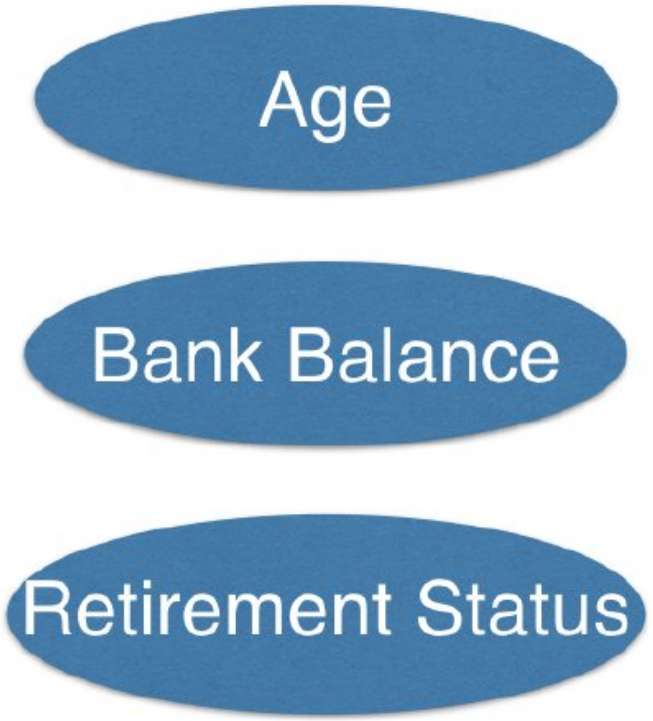
Example as seen by linear regression



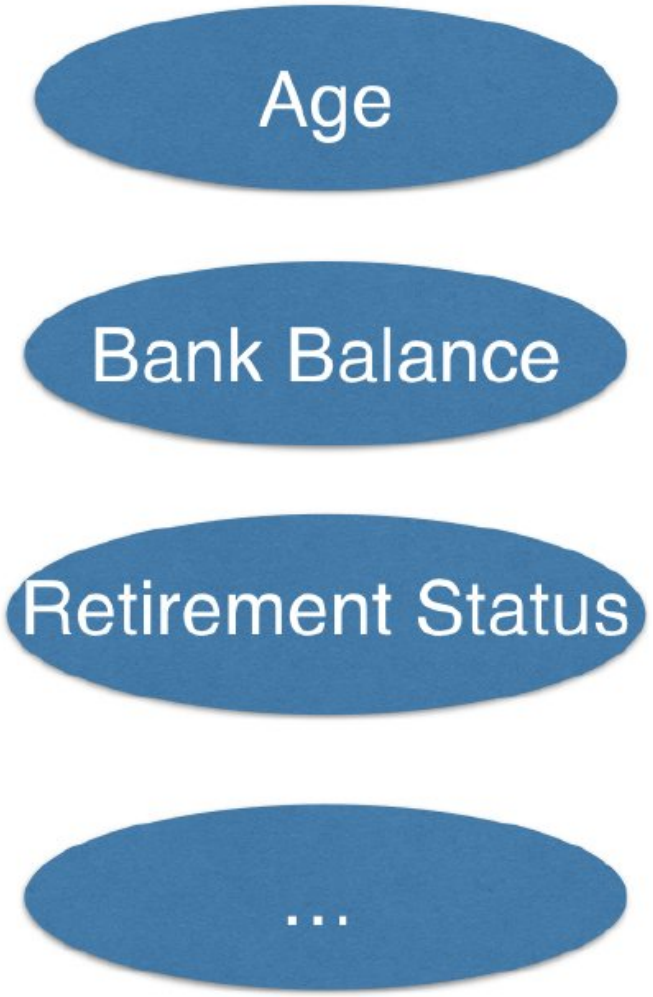
Example as seen by linear regression



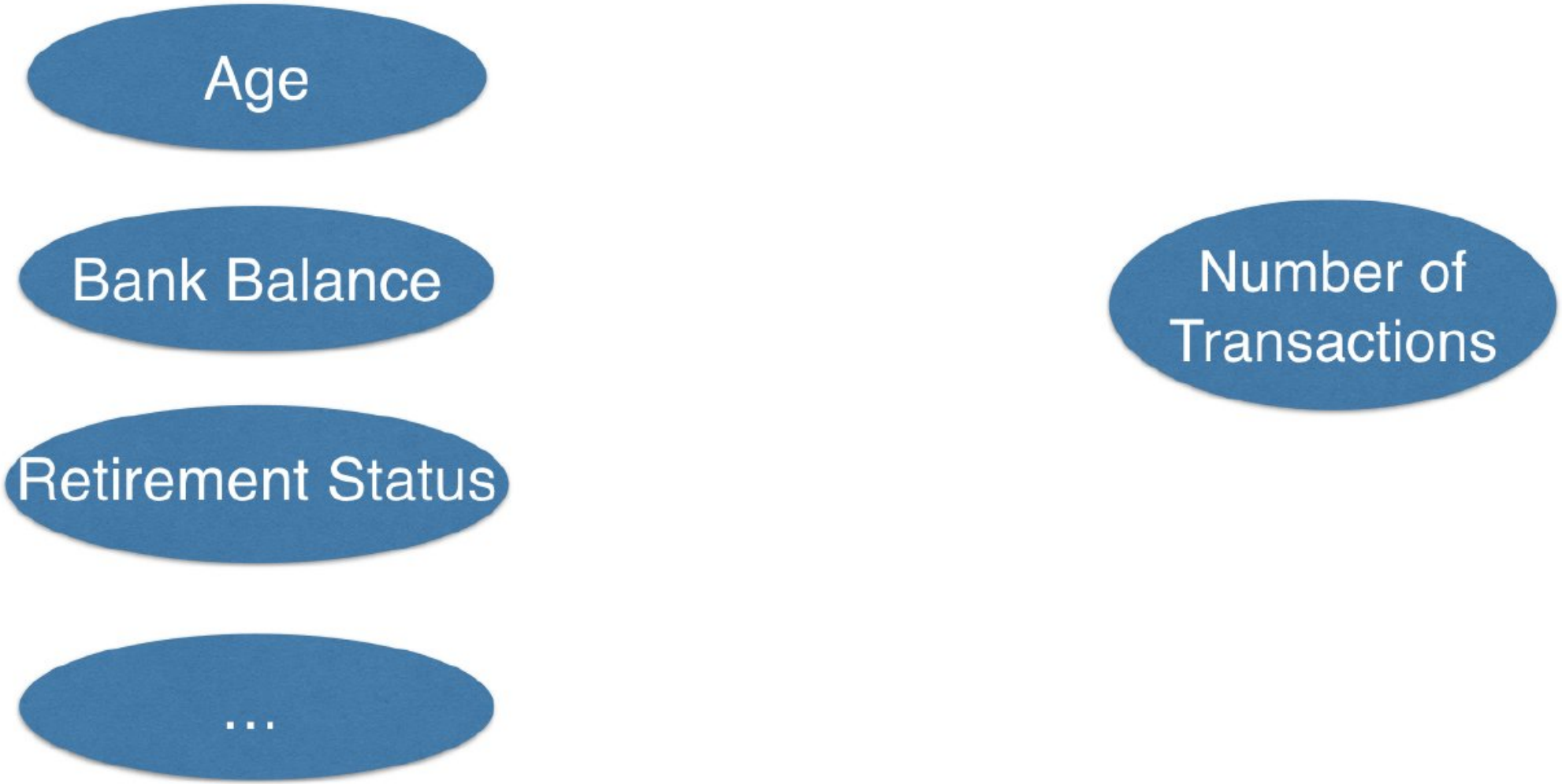
Example as seen by linear regression



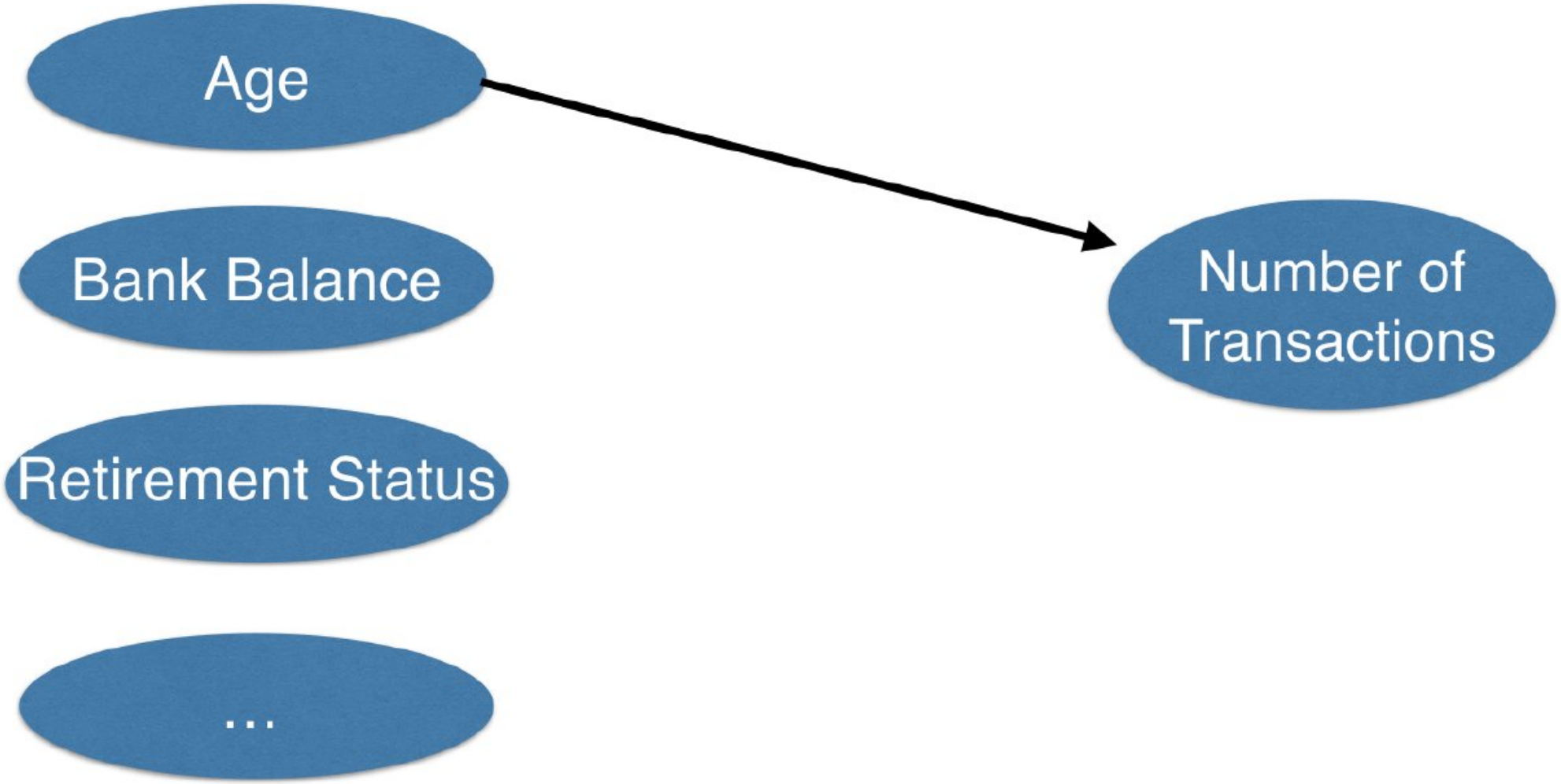
Example as seen by linear regression



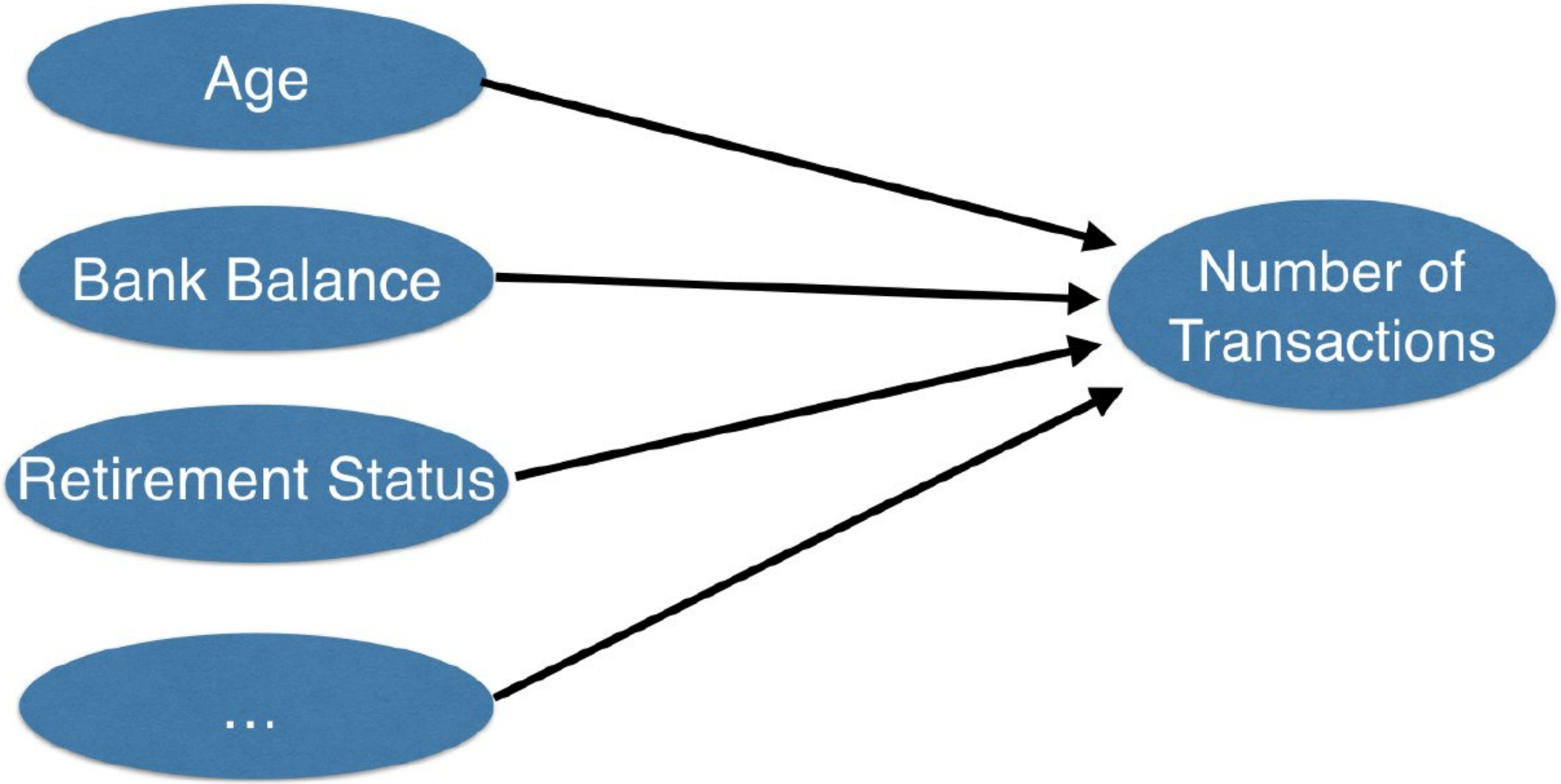
Example as seen by linear regression



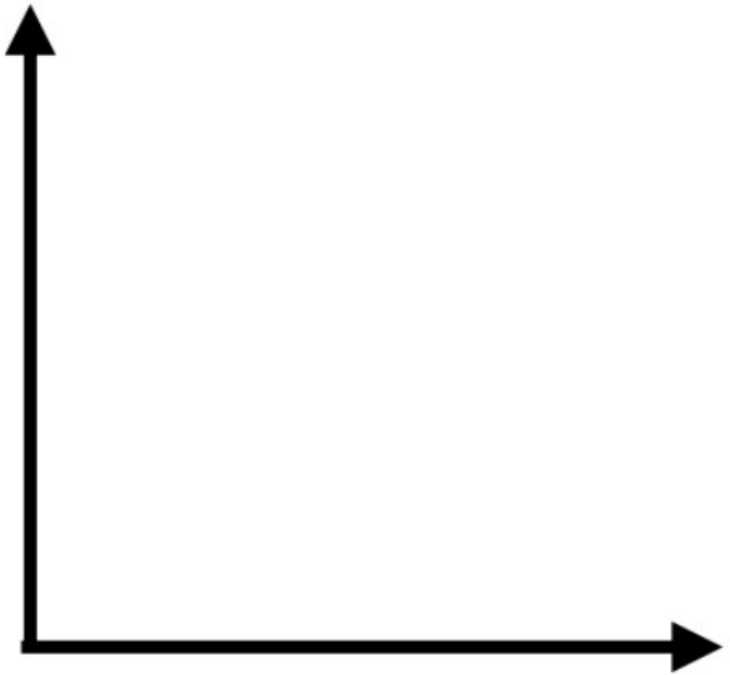
Example as seen by linear regression



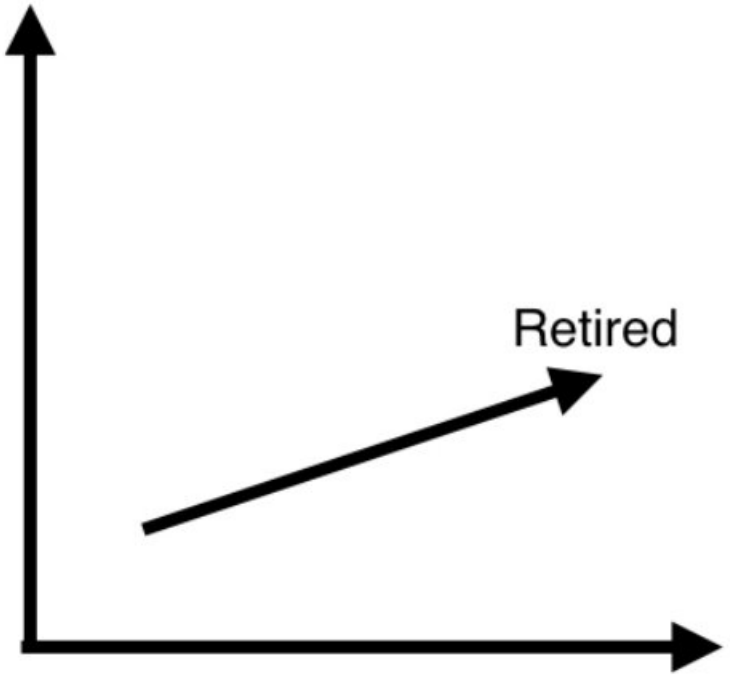
Example as seen by linear regression



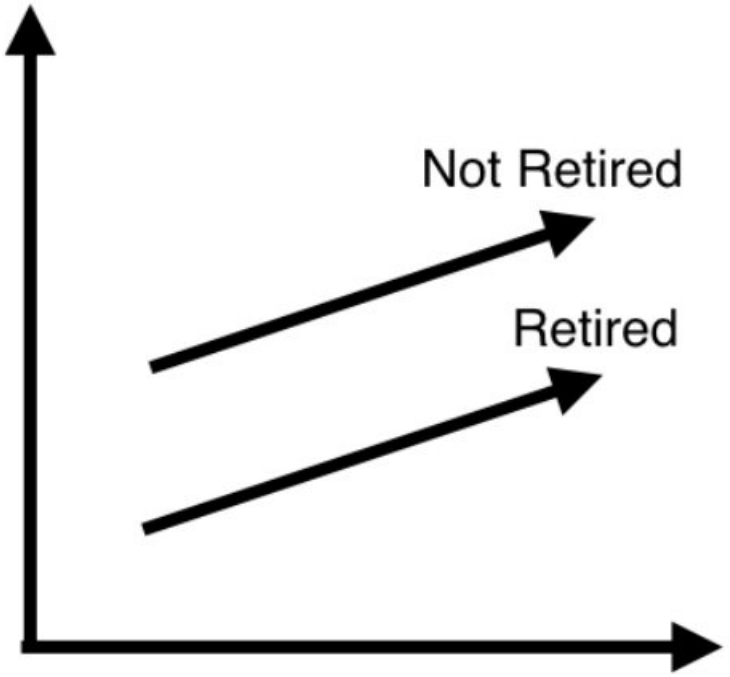
Example as seen by linear regression



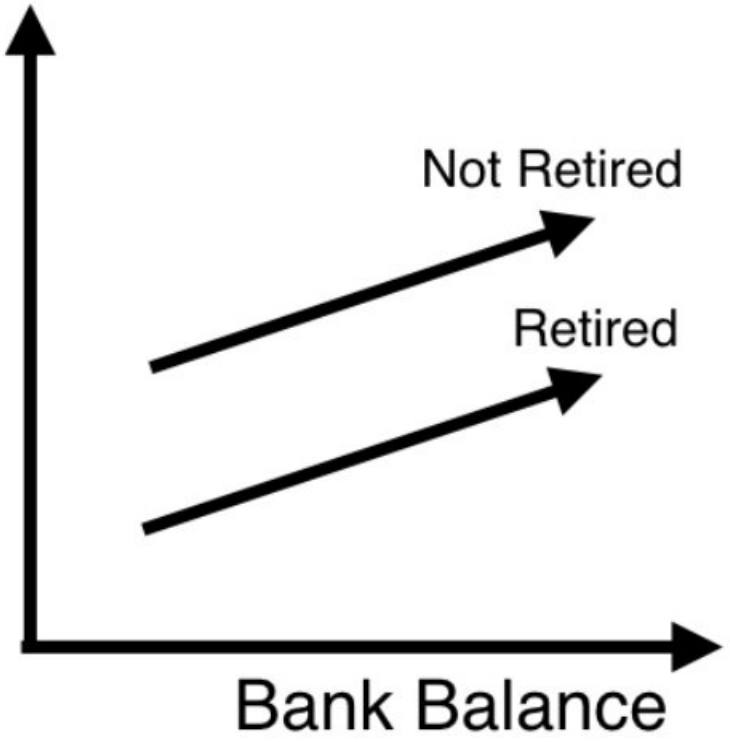
Example as seen by linear regression



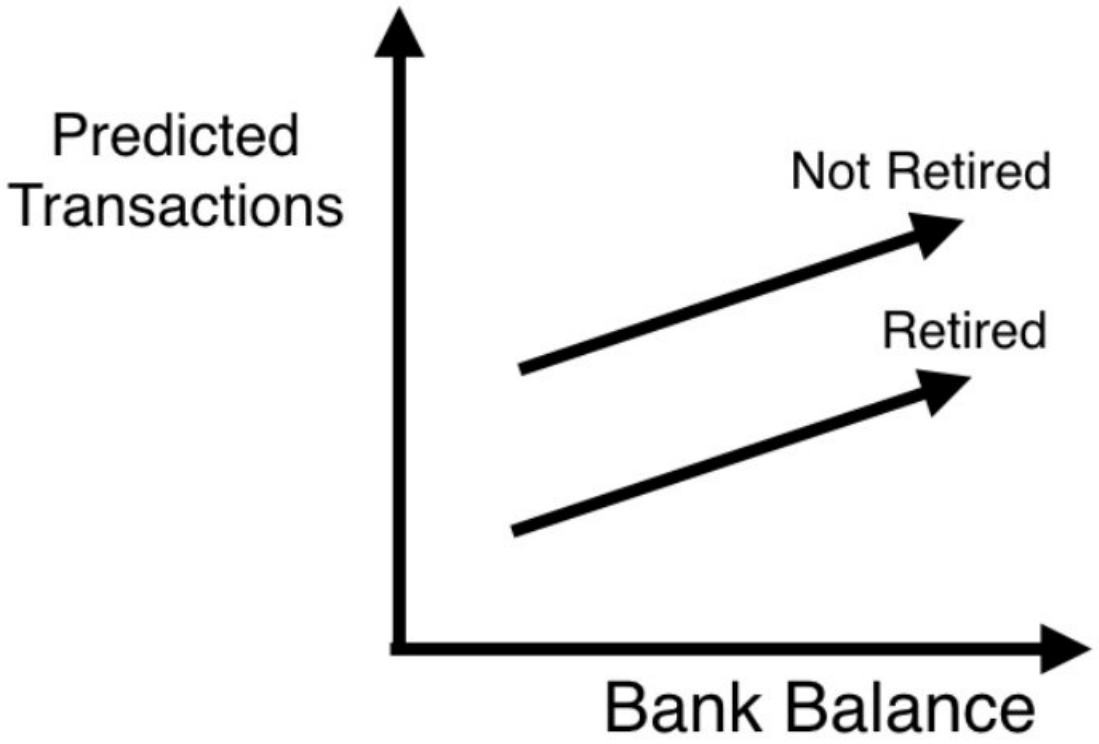
Example as seen by linear regression



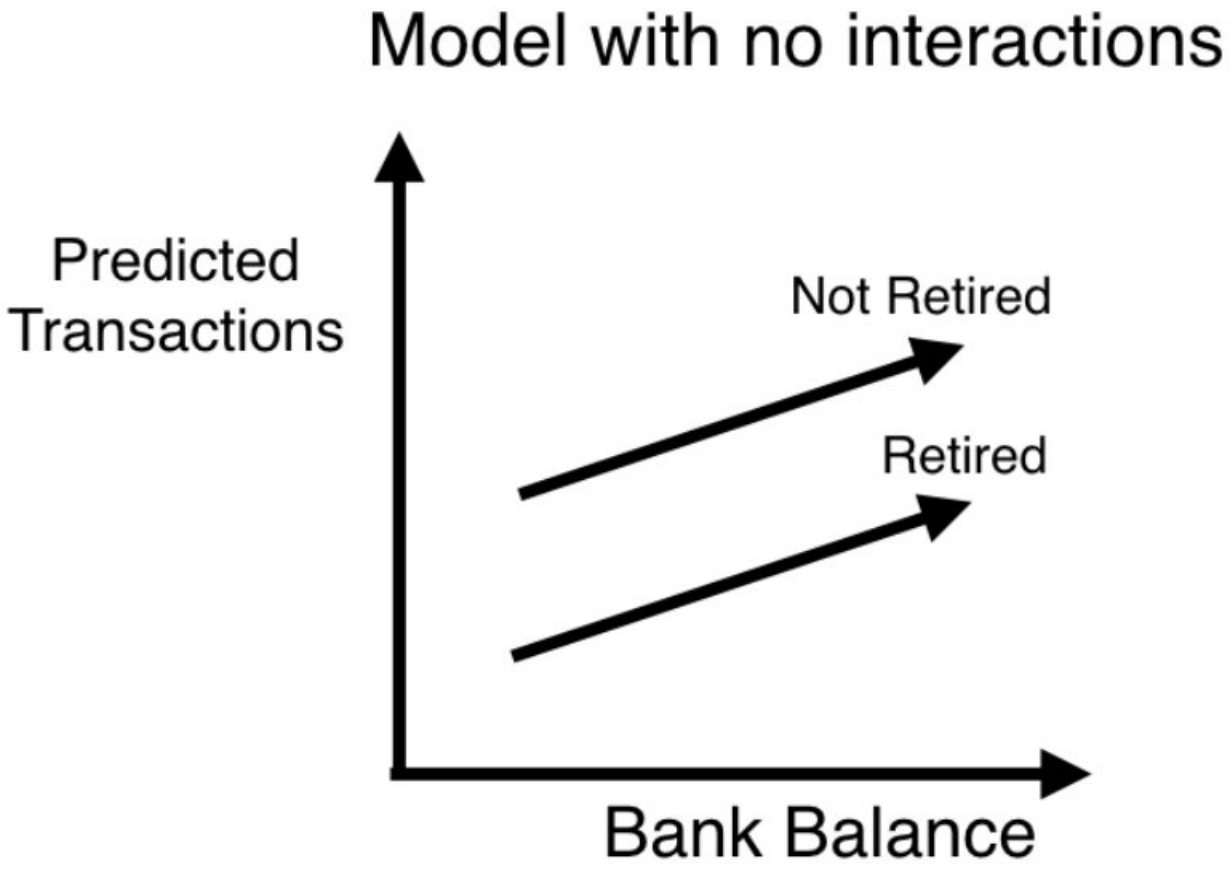
Example as seen by linear regression



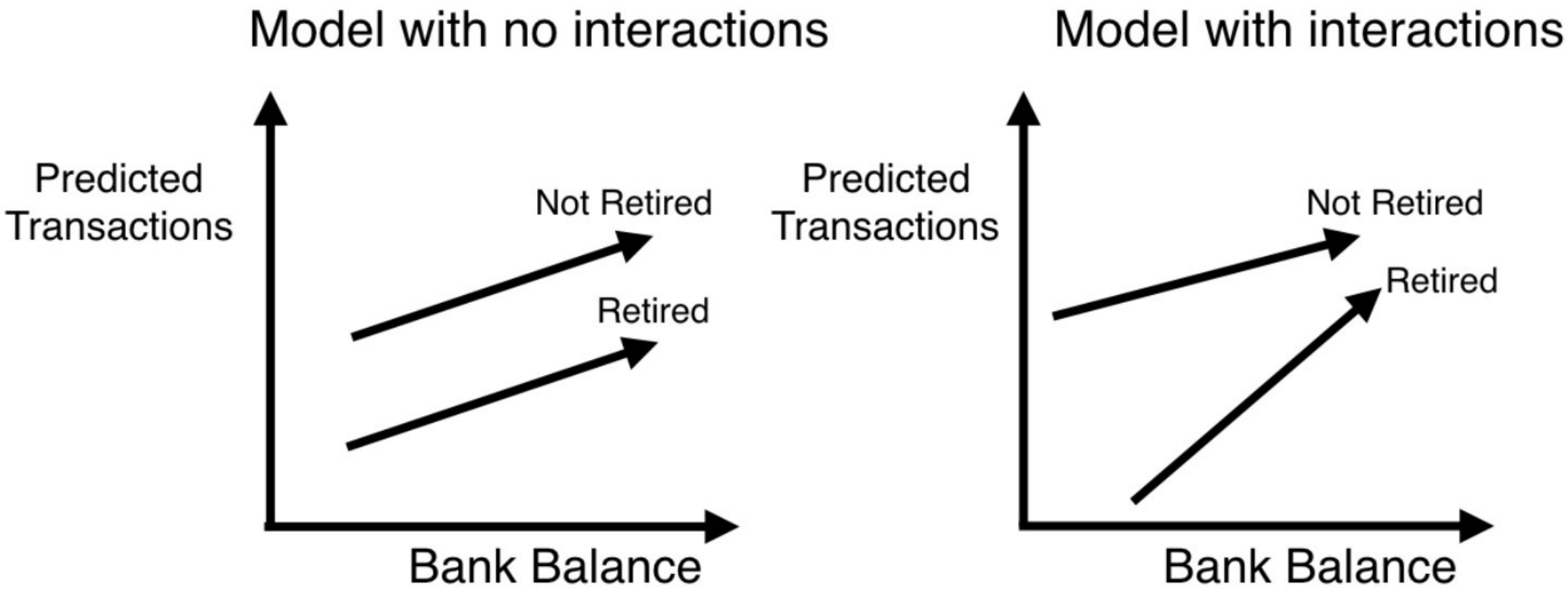
Example as seen by linear regression



Example as seen by linear regression



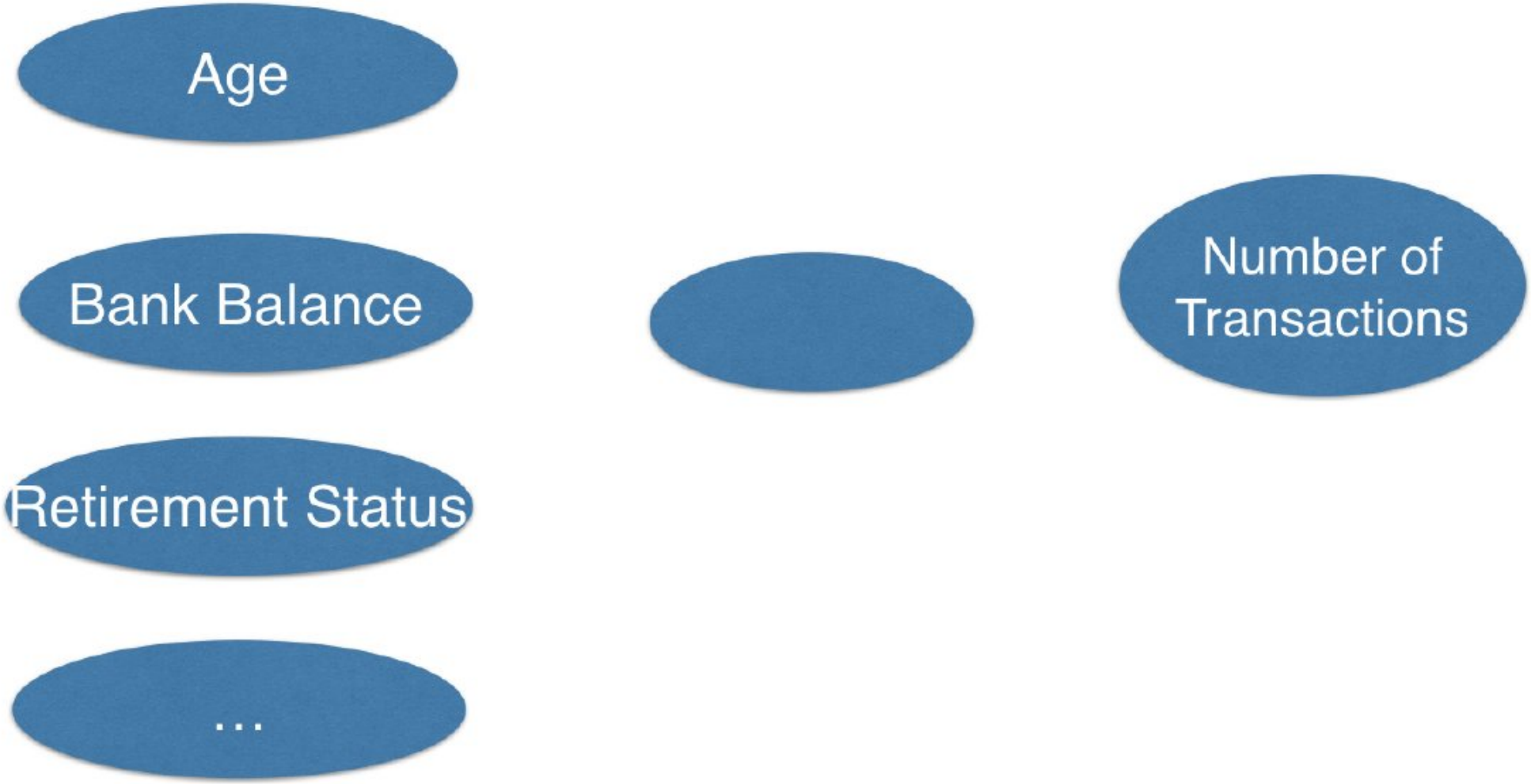
Example as seen by linear regression



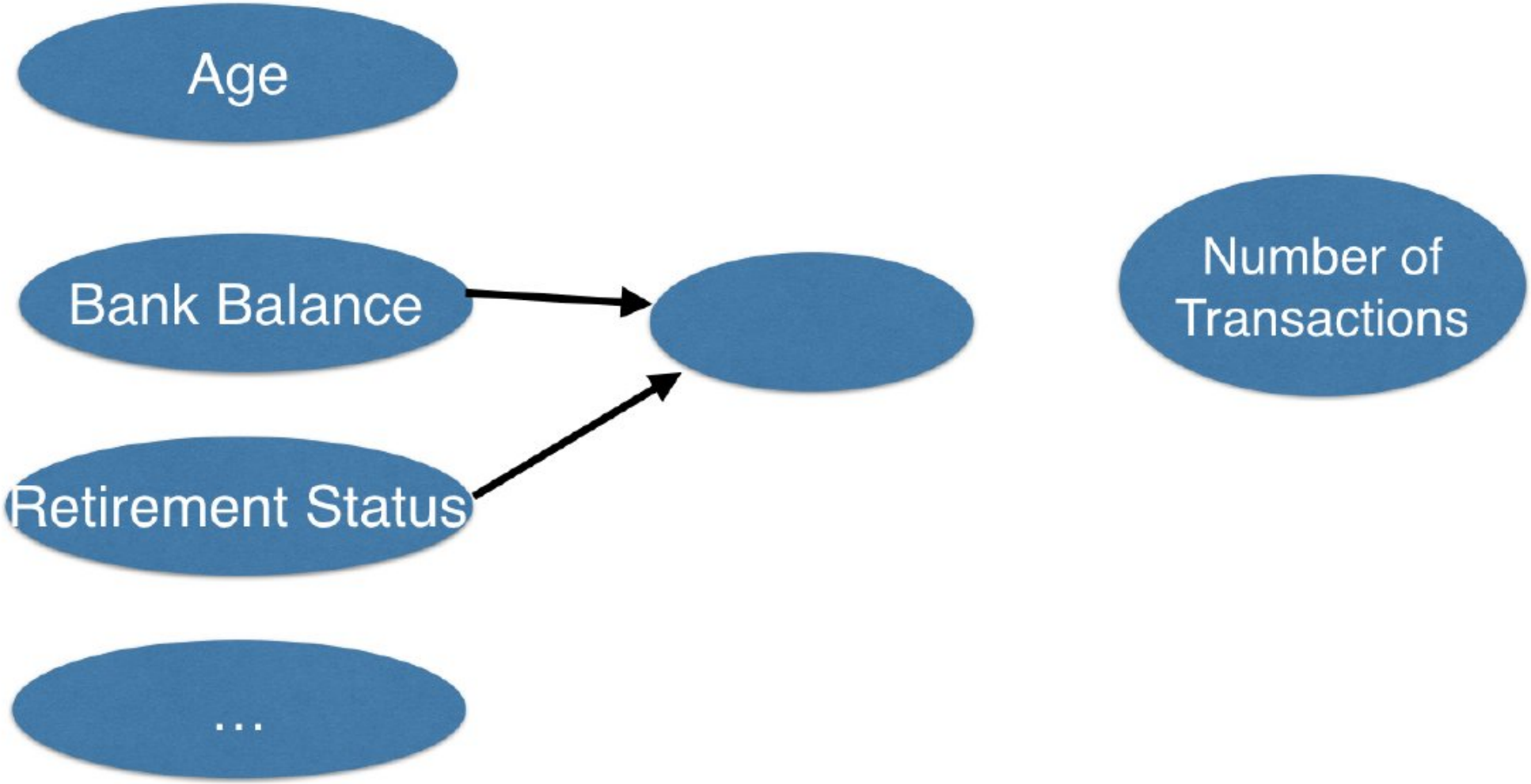
Interactions

- Neural networks account for interactions really well
- Deep learning uses especially powerful neural networks
  - Text
  - Images
  - Videos
  - Audio
  - Source code

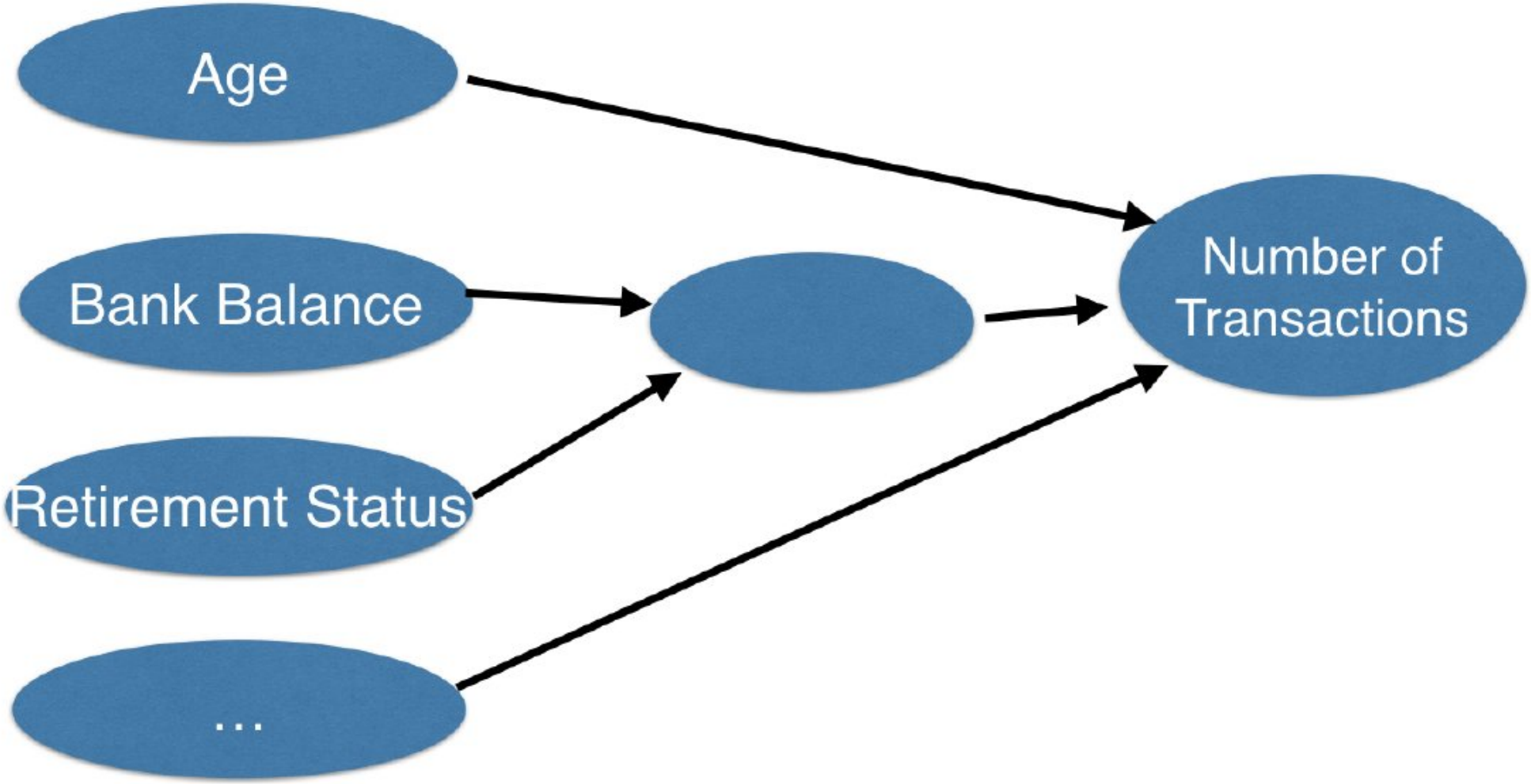
Deep learning models capture interactions



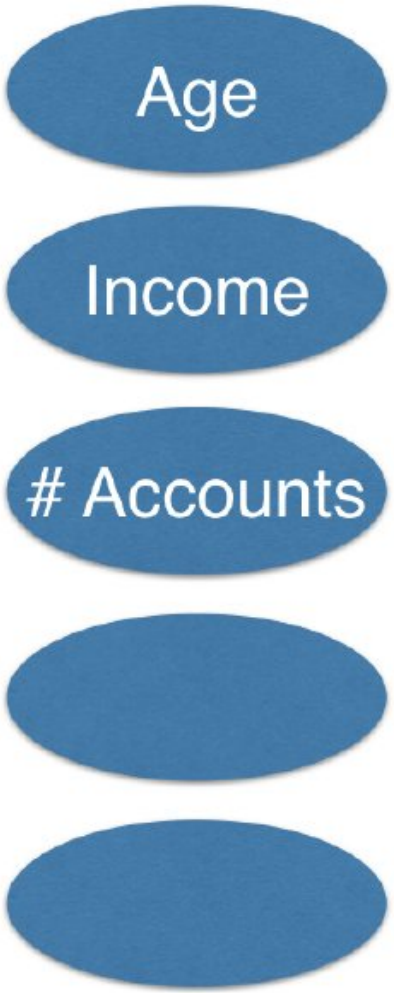
Deep learning models capture interactions



Deep learning models capture interactions



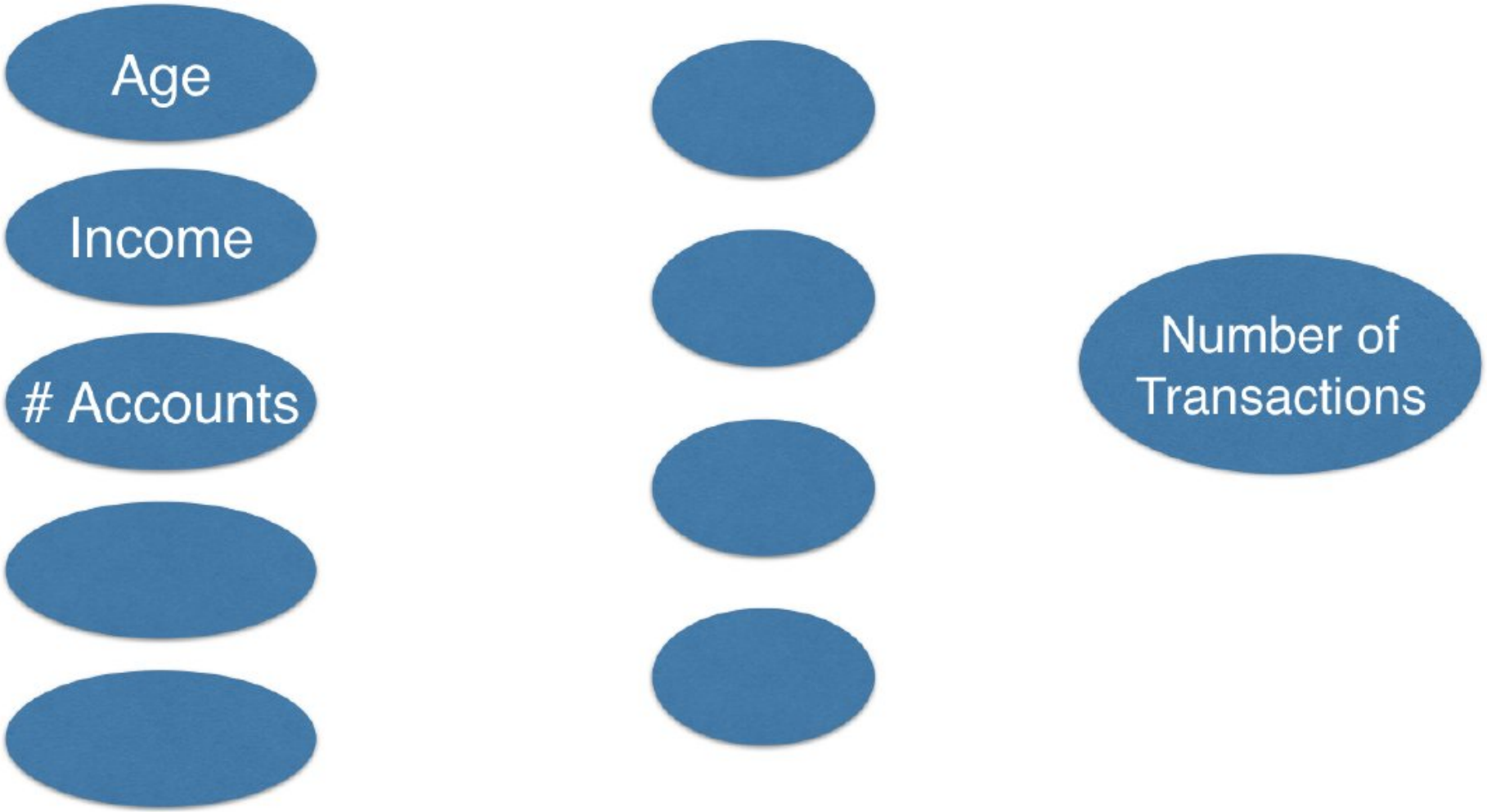
Interactions in neural network



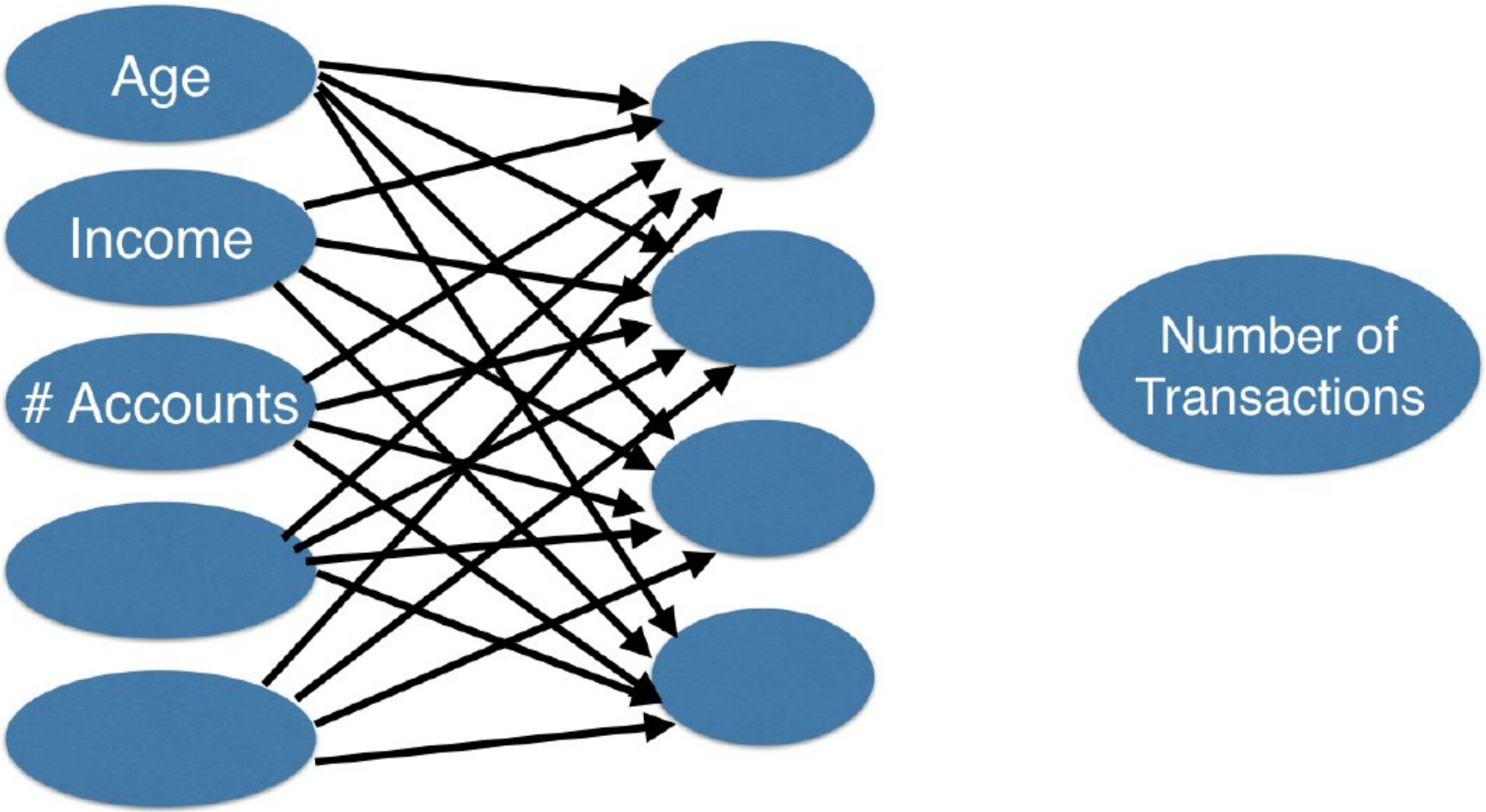
Interactions in neural network



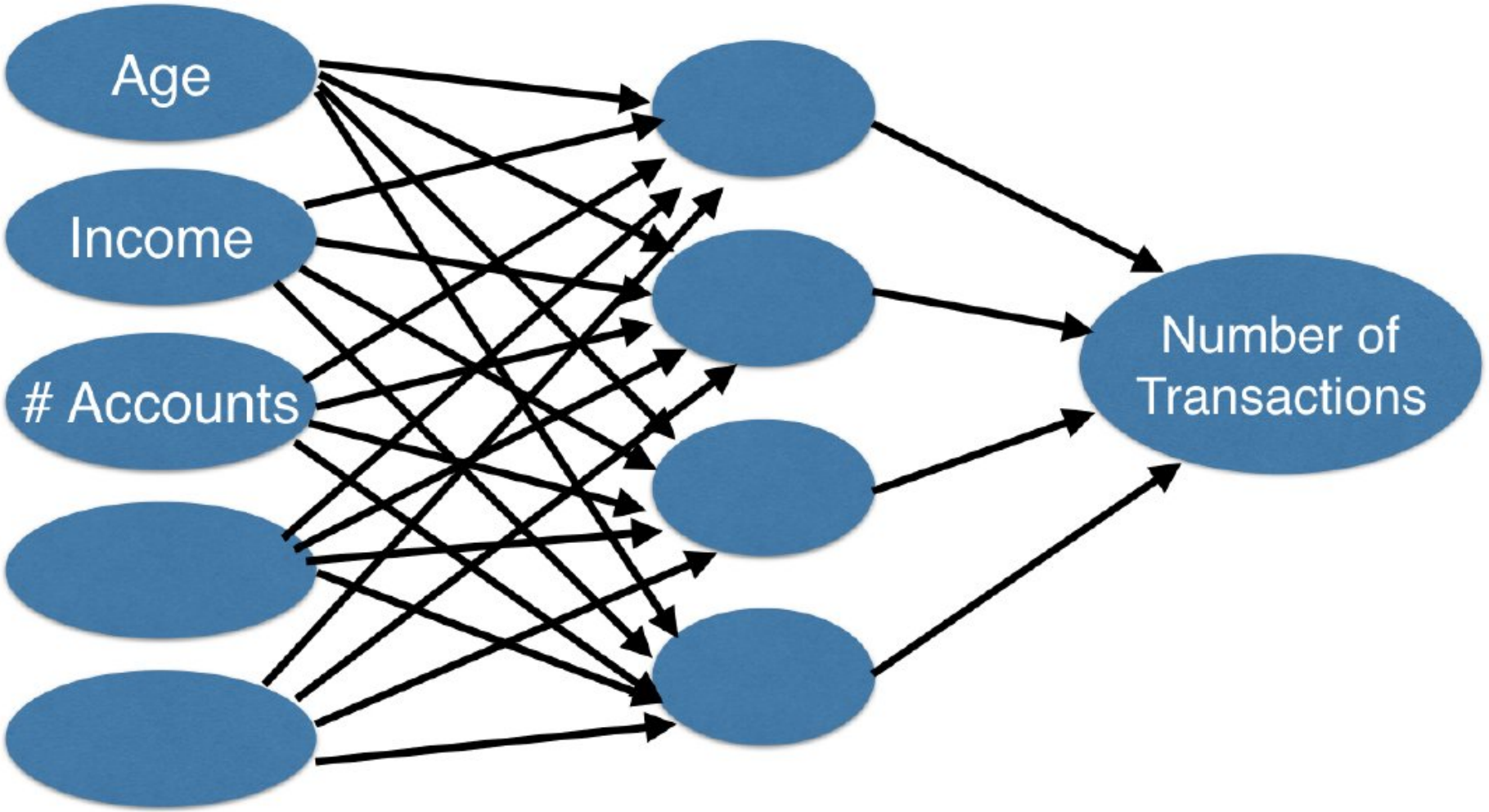
Interactions in neural network



Interactions in neural network



Interactions in neural network



Forward propagation

INTRODUCTION TO DEEP LEARNING IN PYTHON

Dan Becker

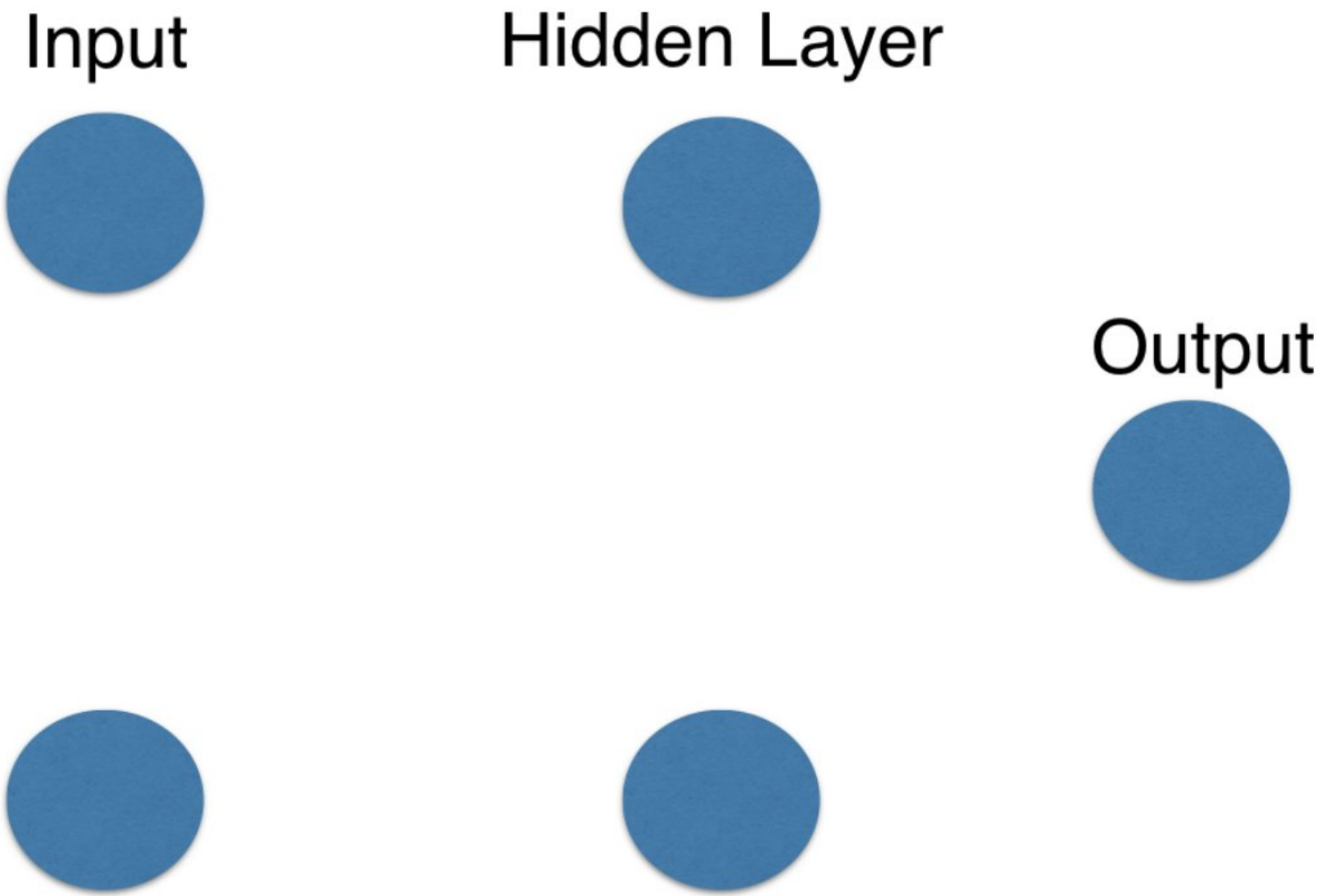
Data Scientist and contributor to Keras and TensorFlow libraries



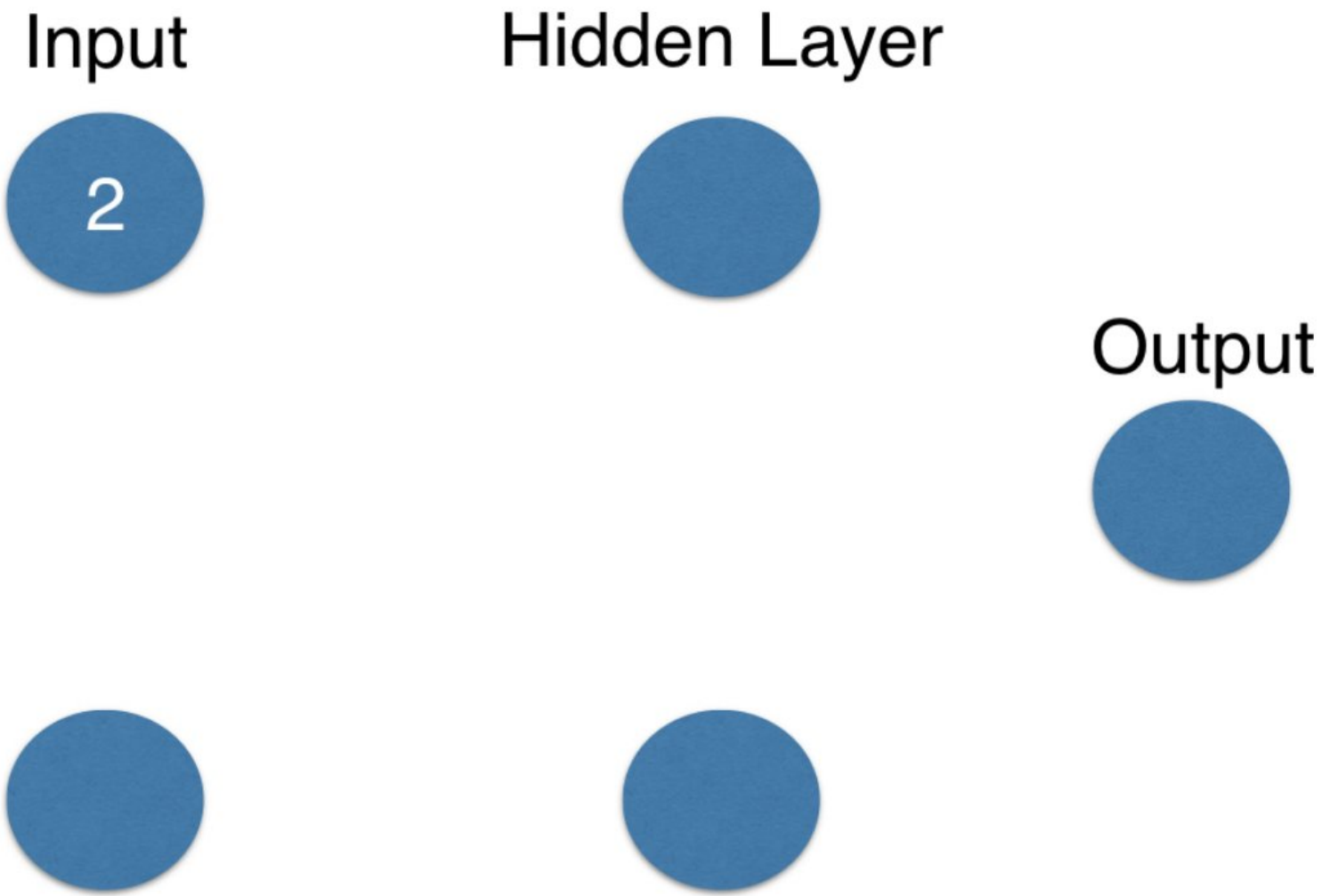
Bank transactions example

- Make predictions based on:
  - Number of children
  - Number of existing accounts

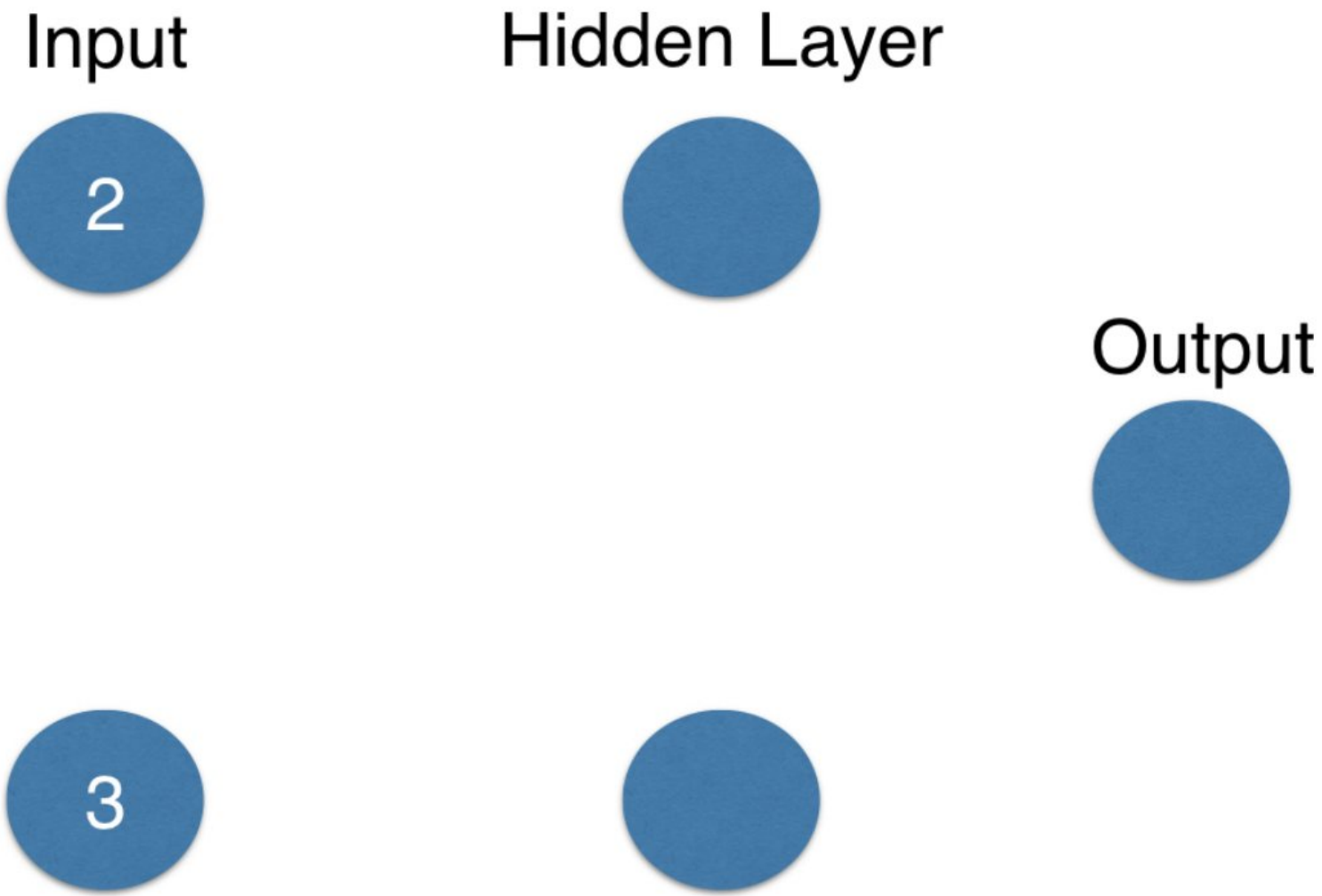
Forward propagation



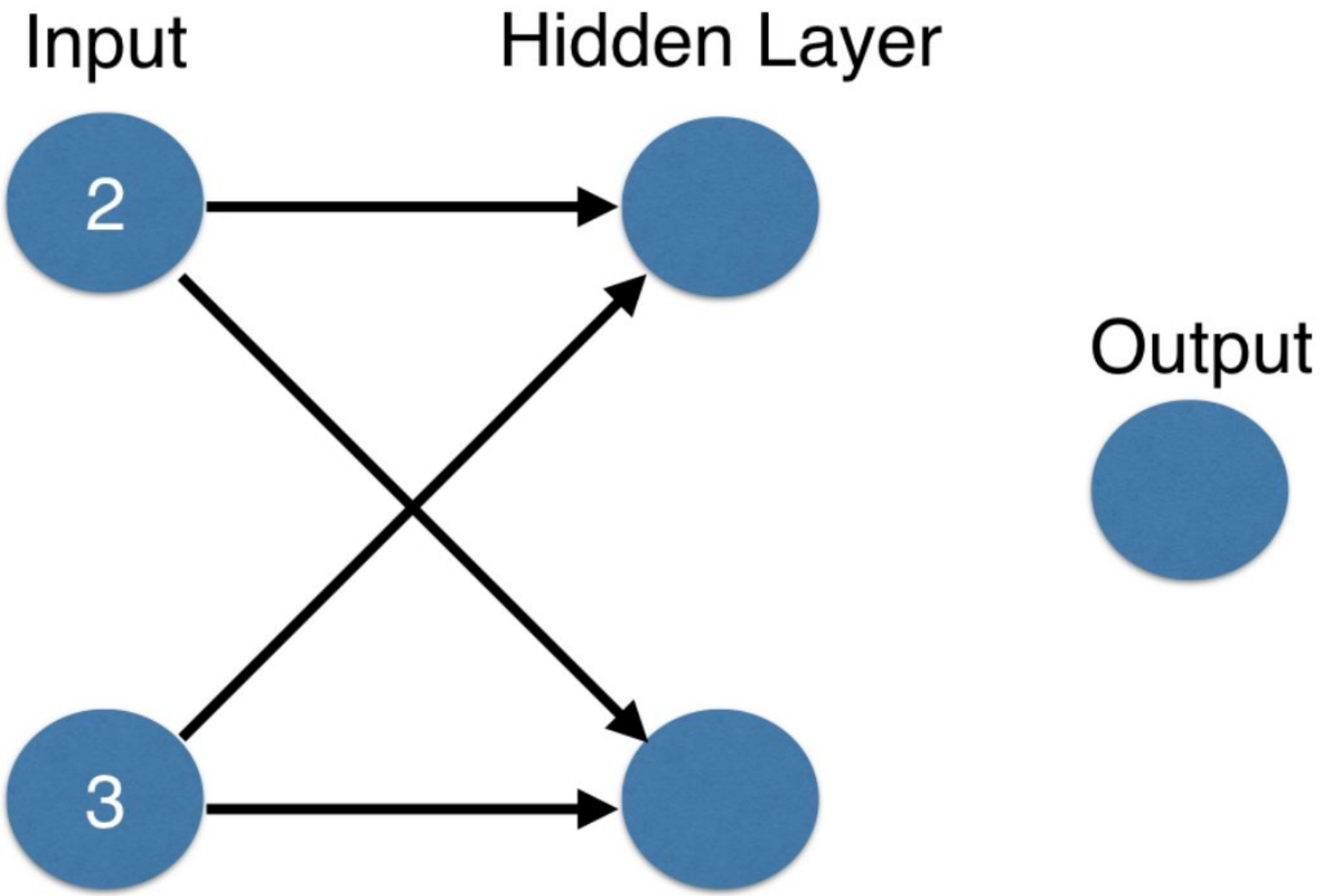
Forward propagation



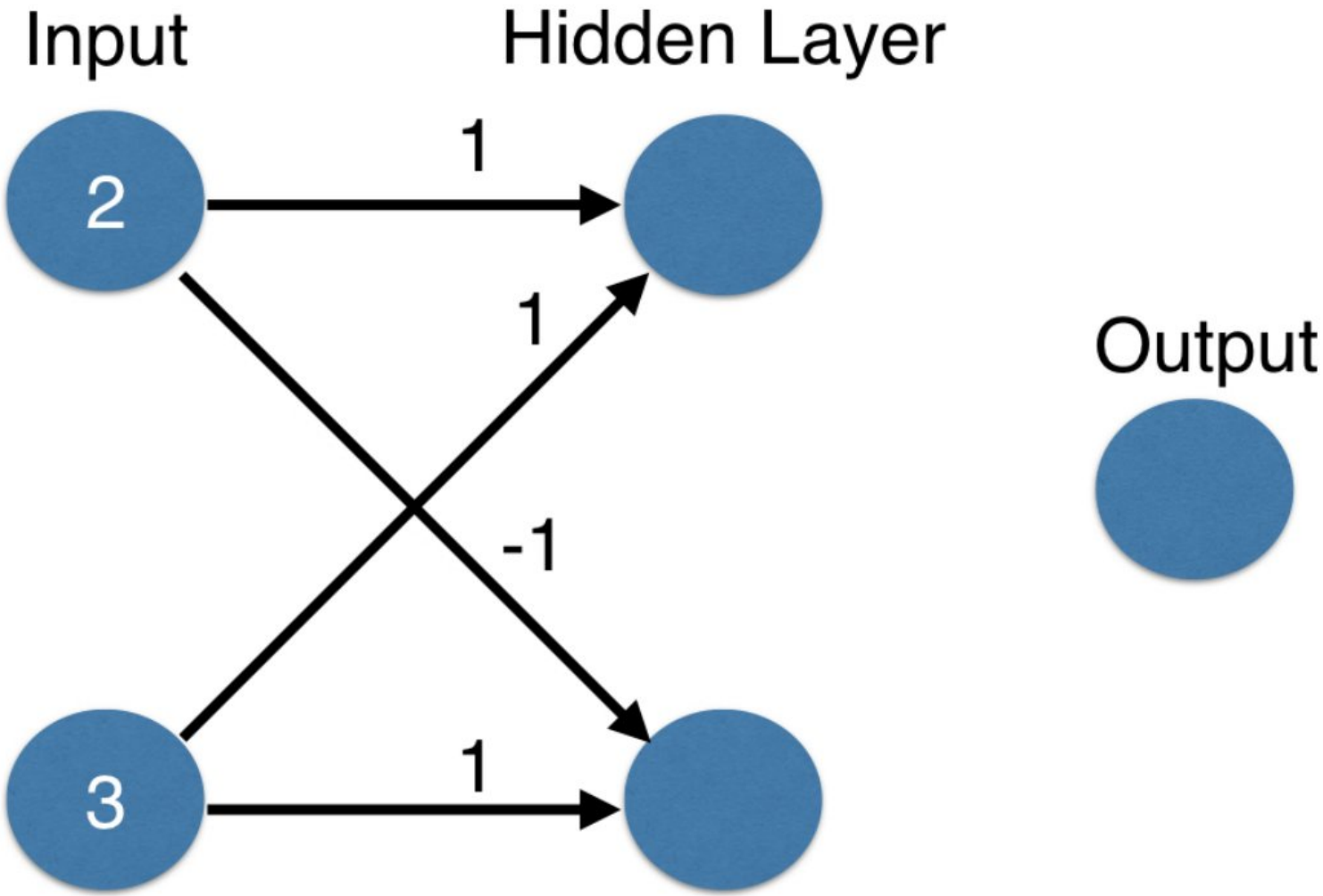
Forward propagation



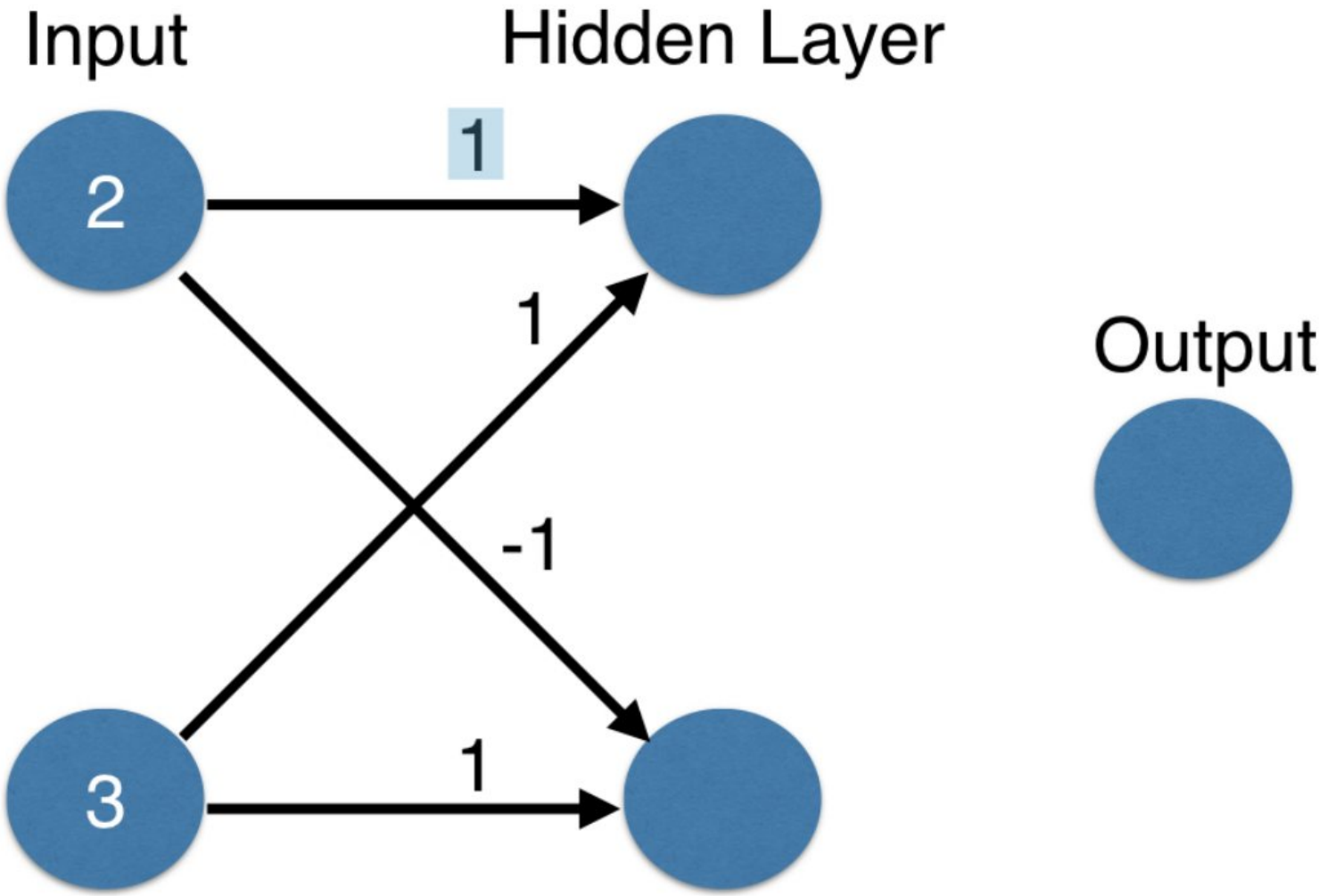
Forward propagation



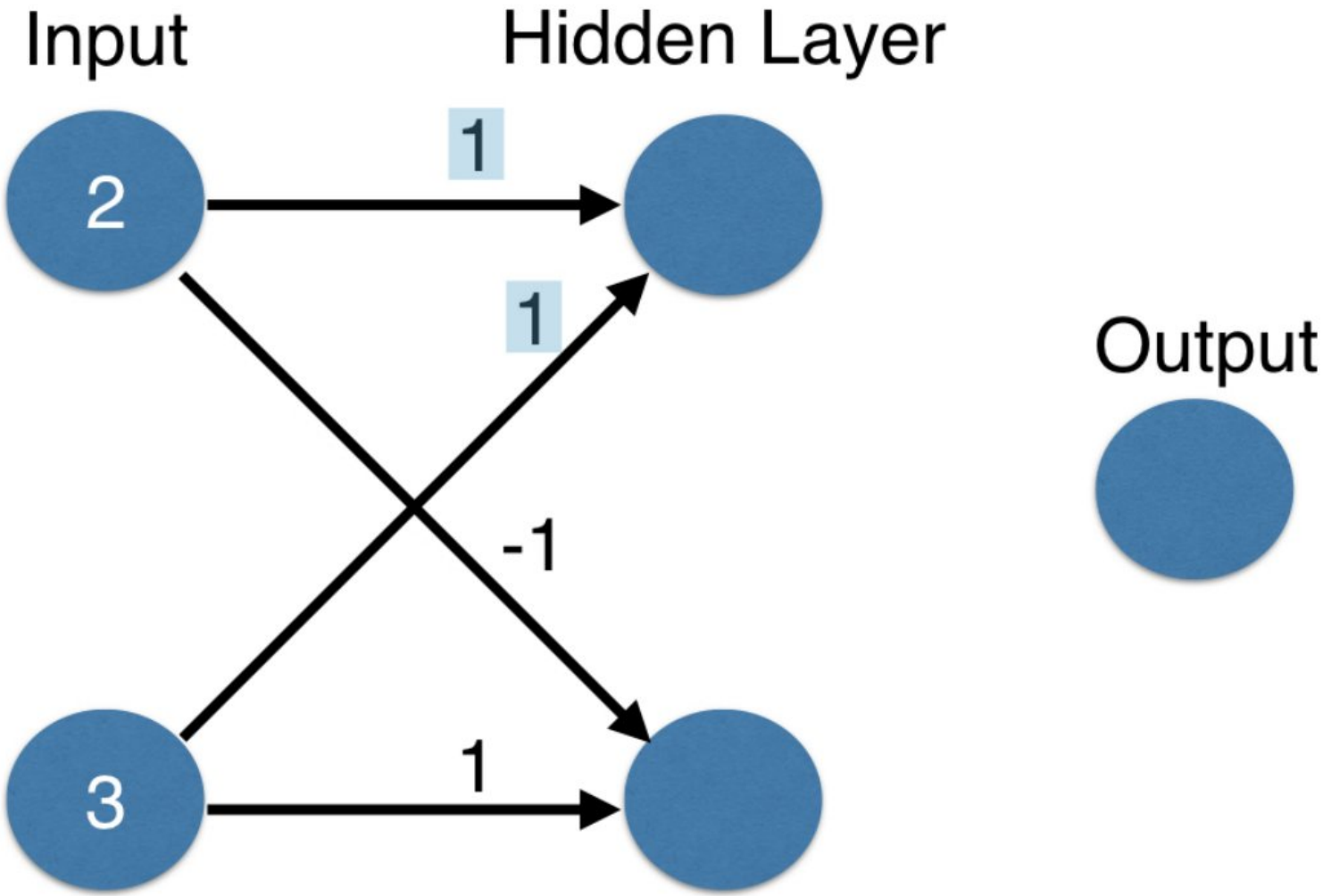
Forward propagation



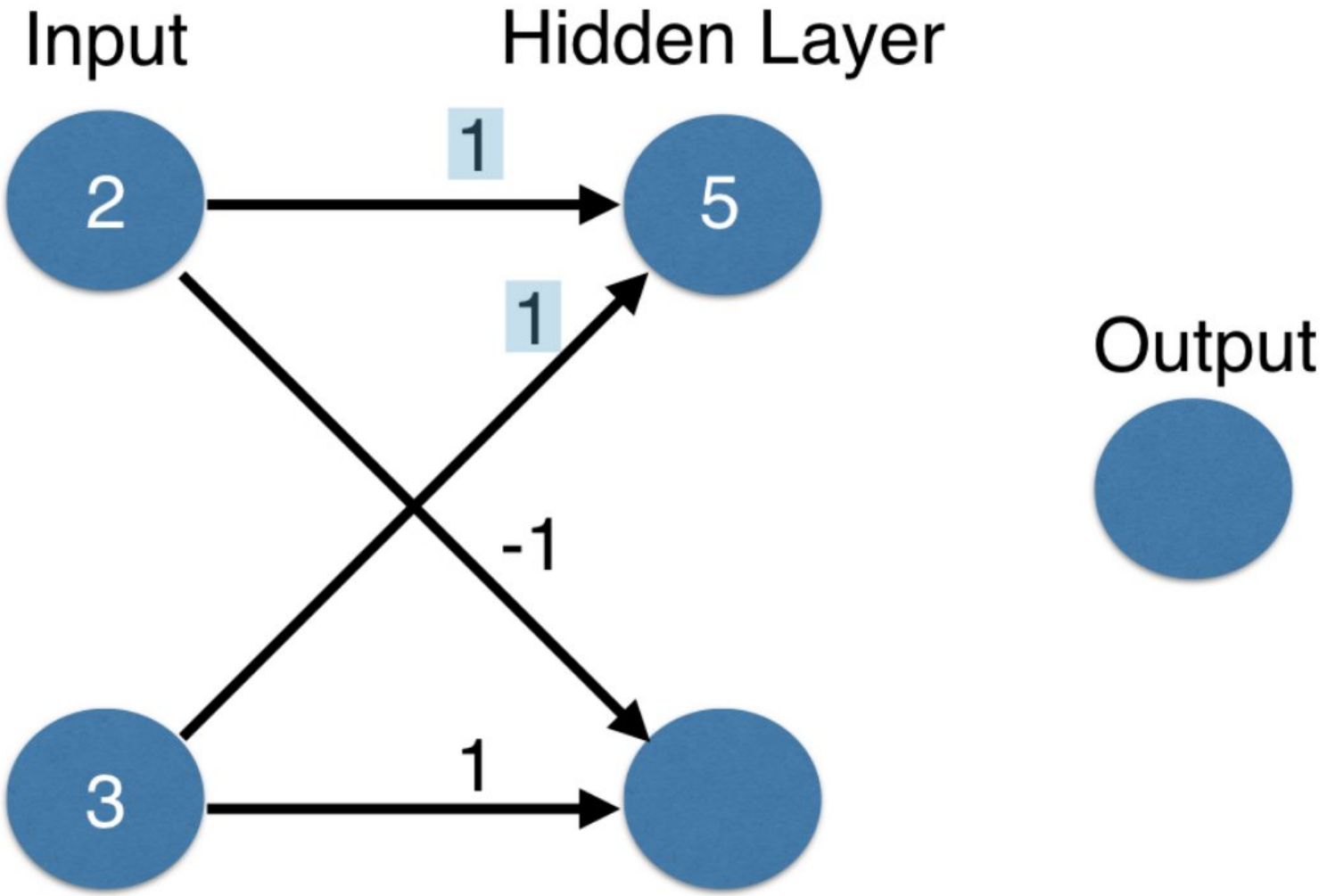
Forward propagation



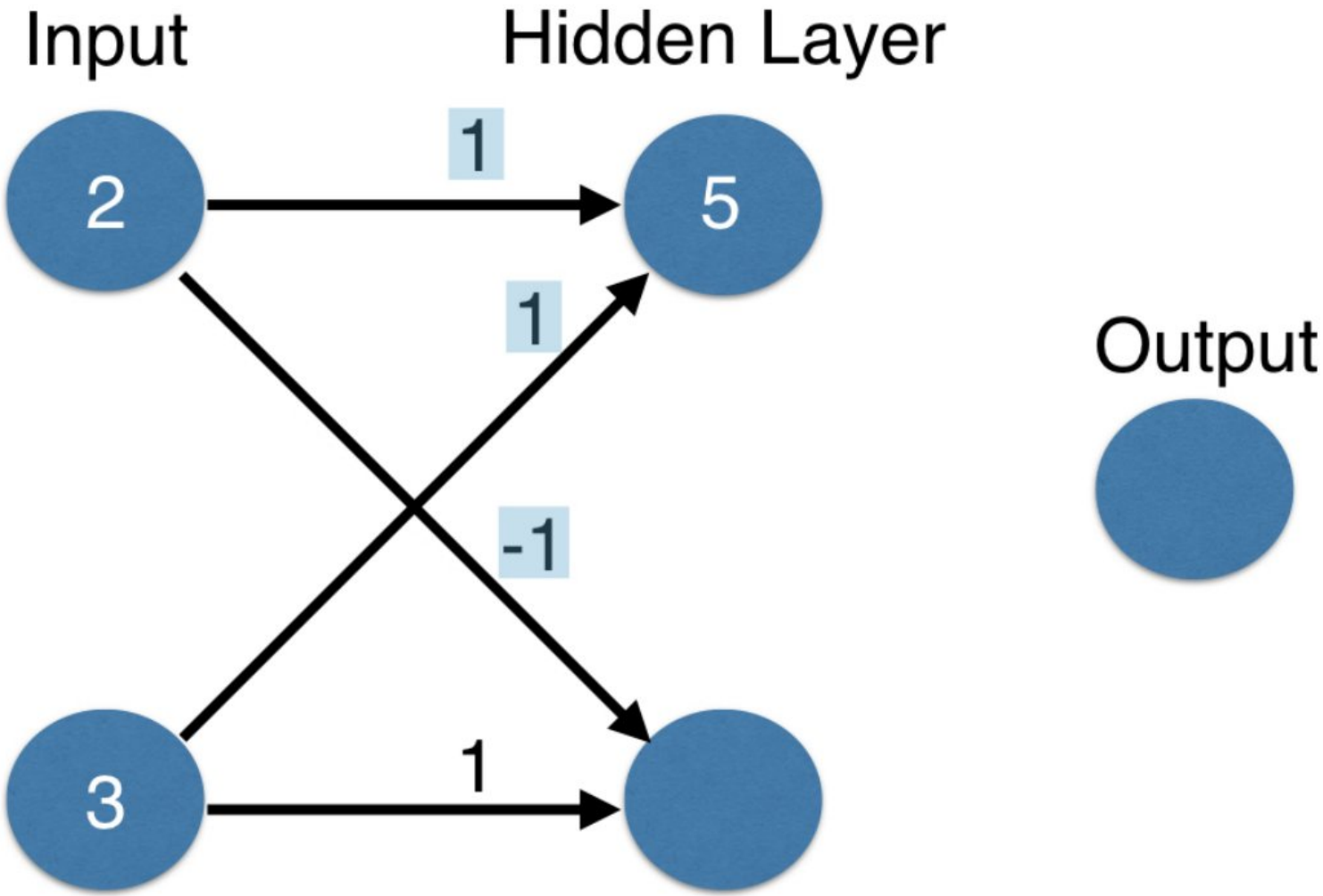
Forward propagation



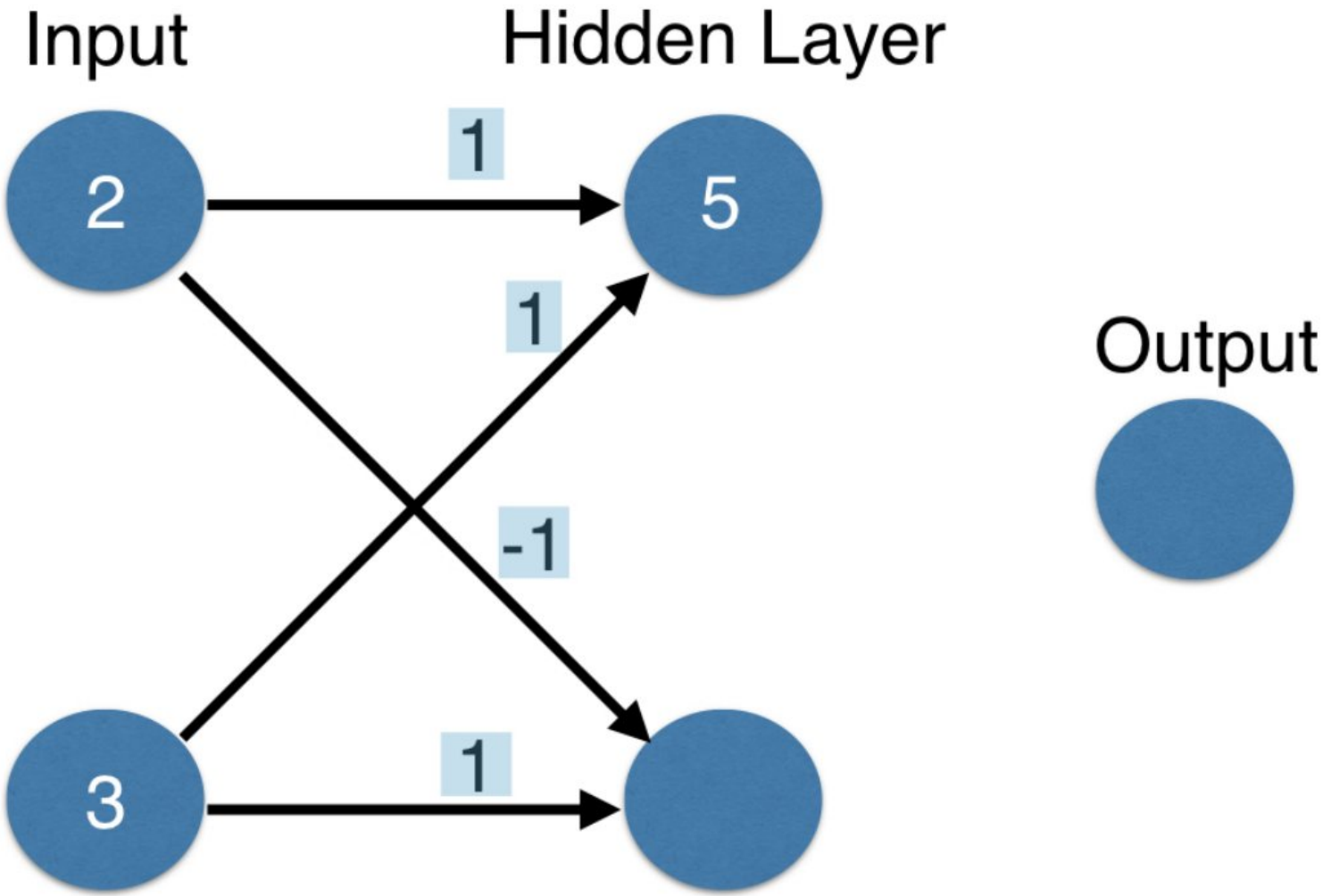
Forward propagation



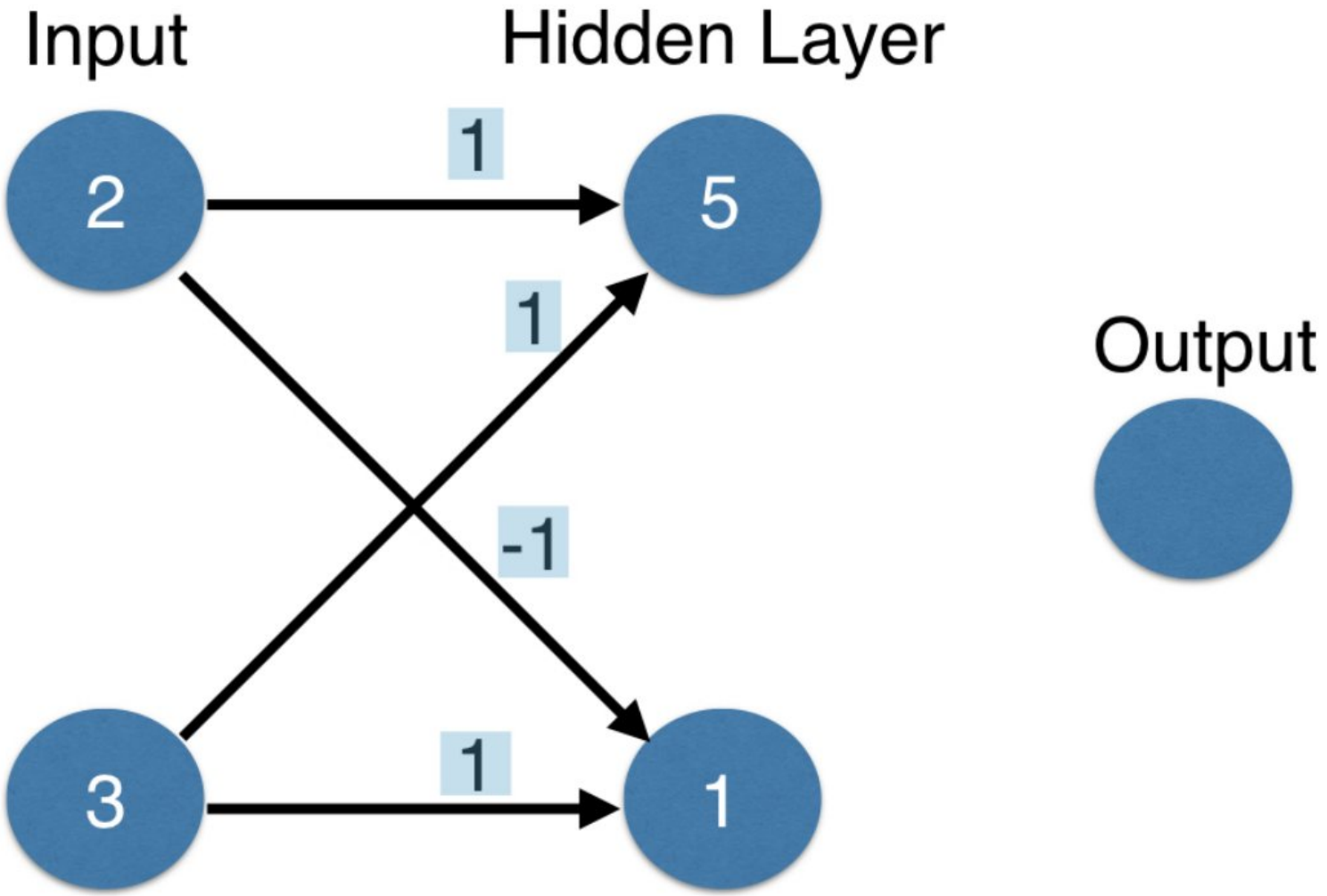
Forward propagation



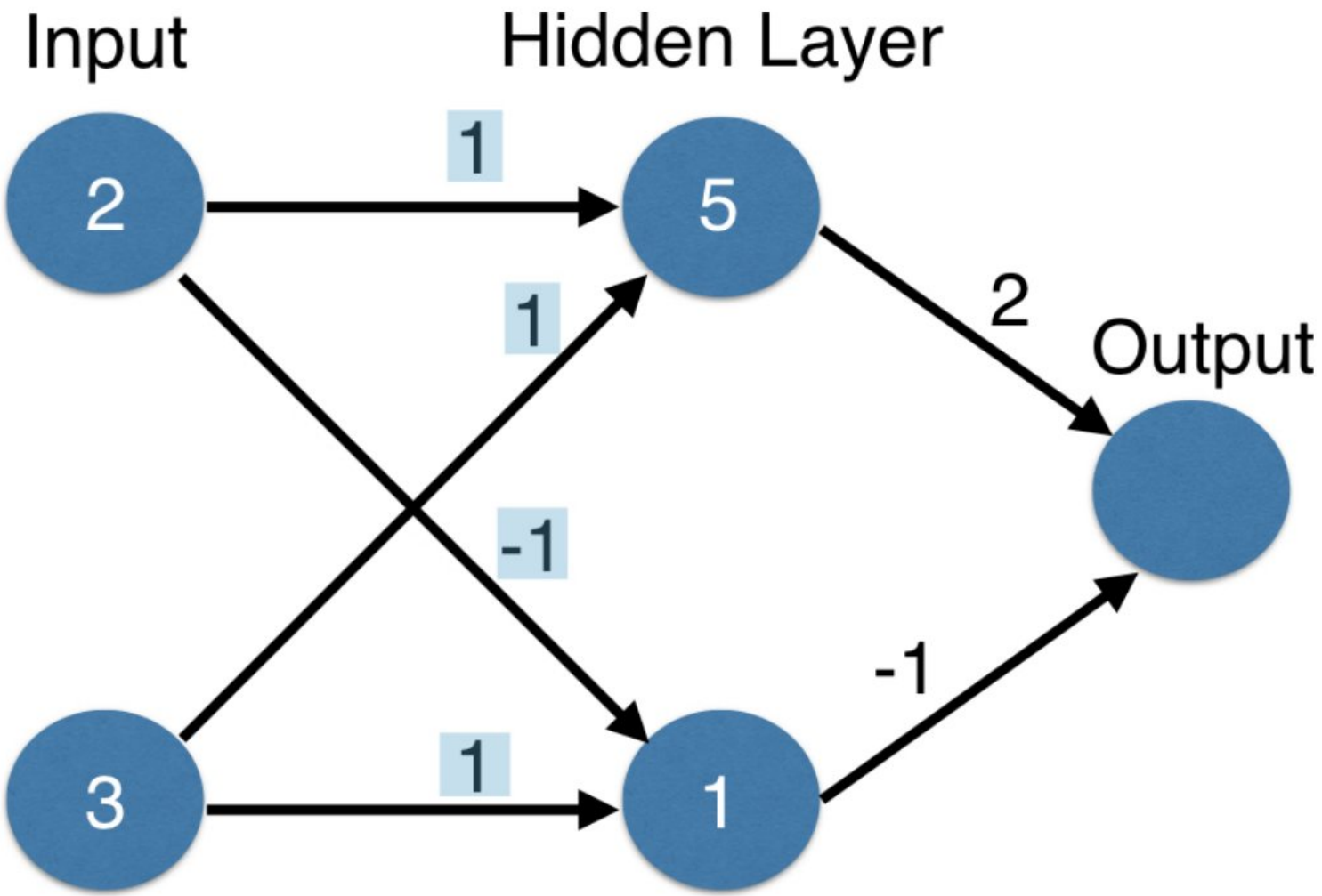
Forward propagation



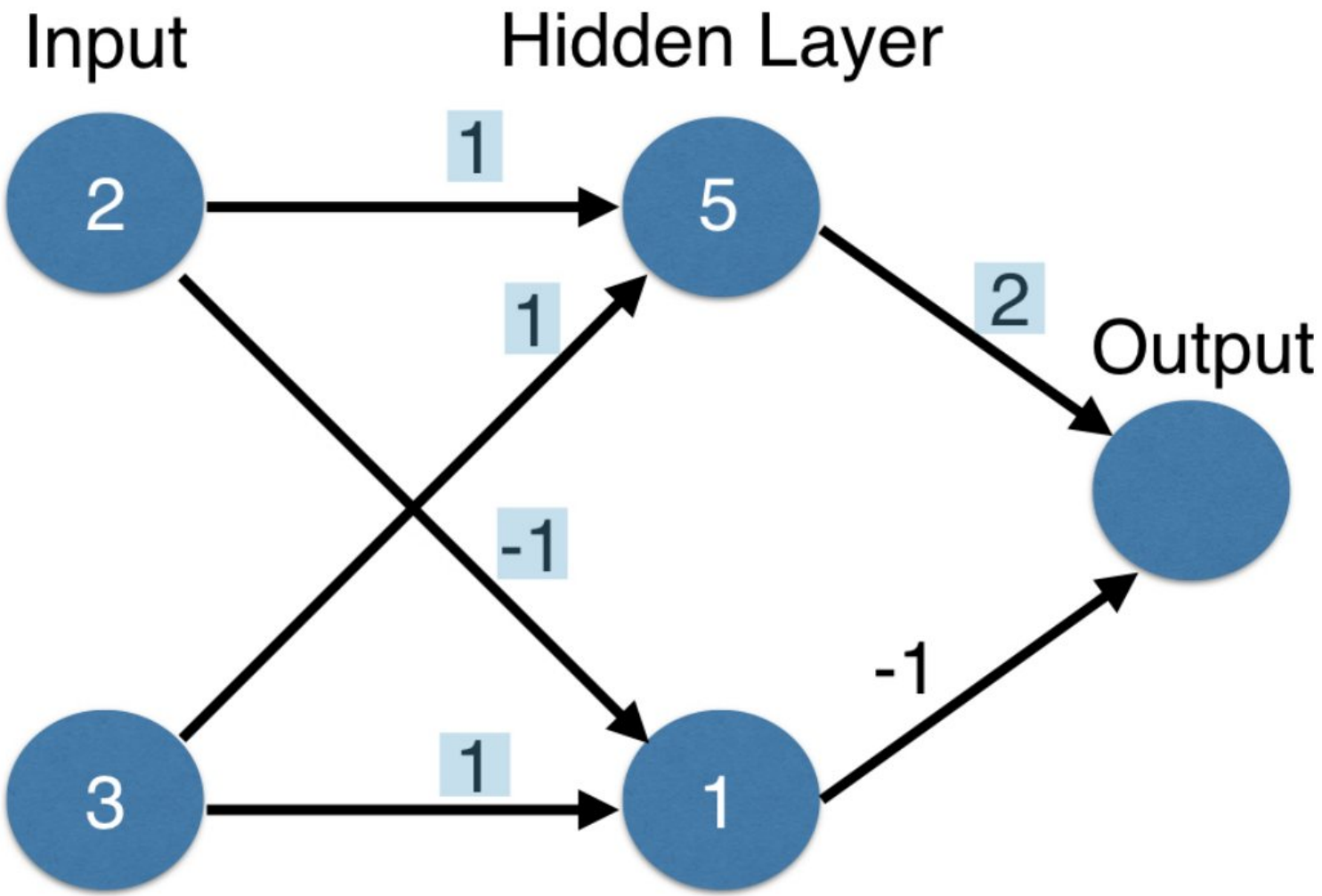
Forward propagation



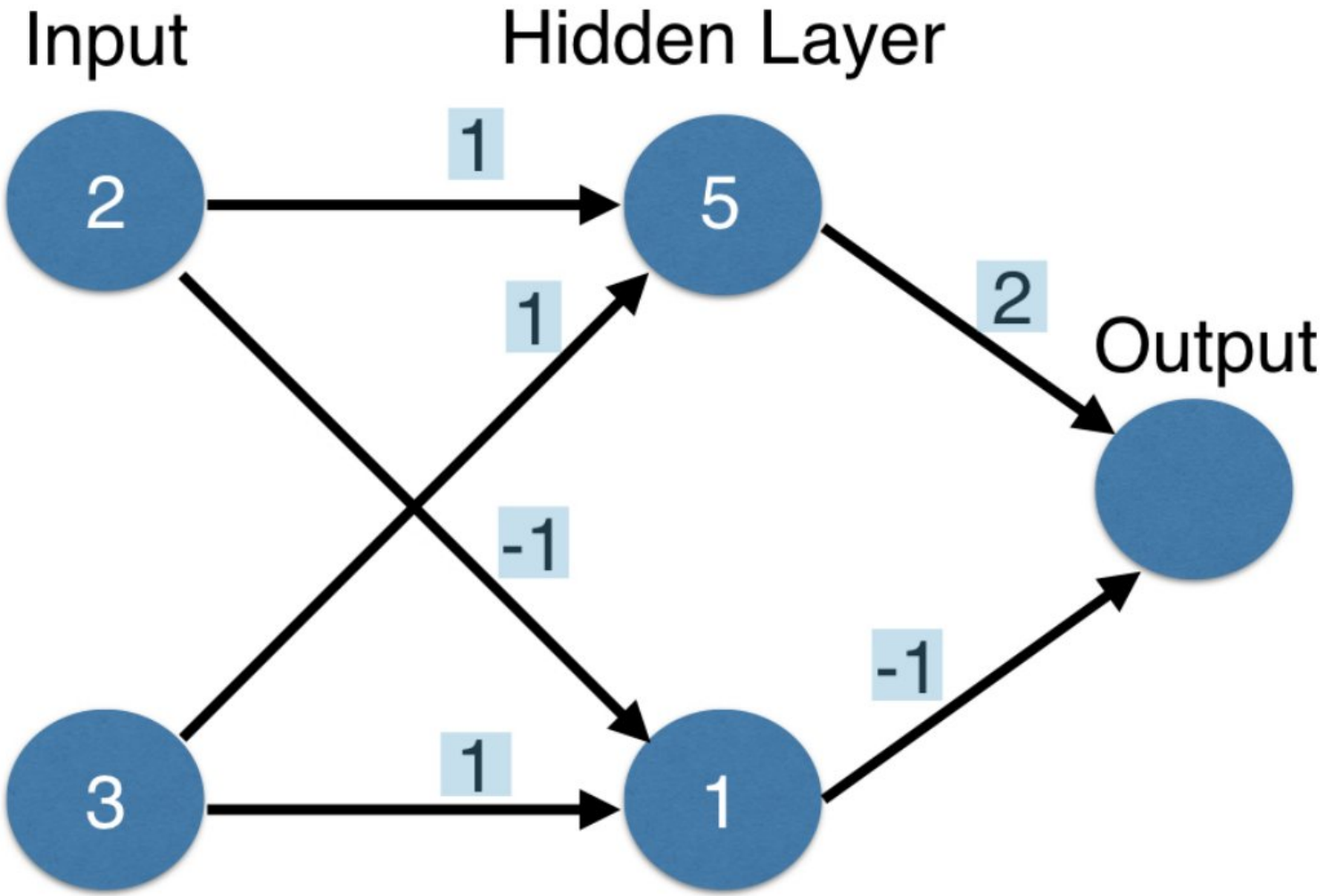
Forward propagation



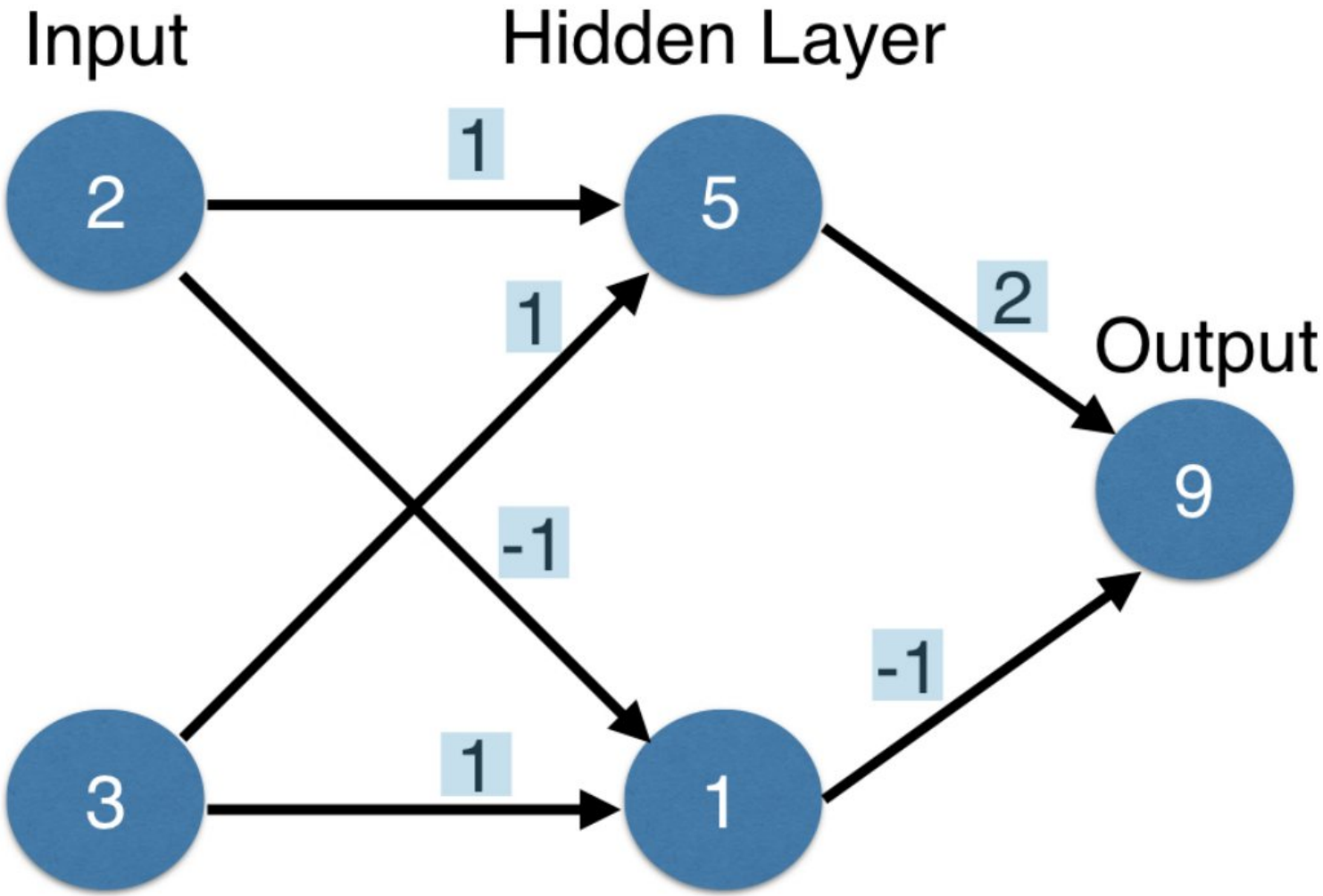
Forward propagation



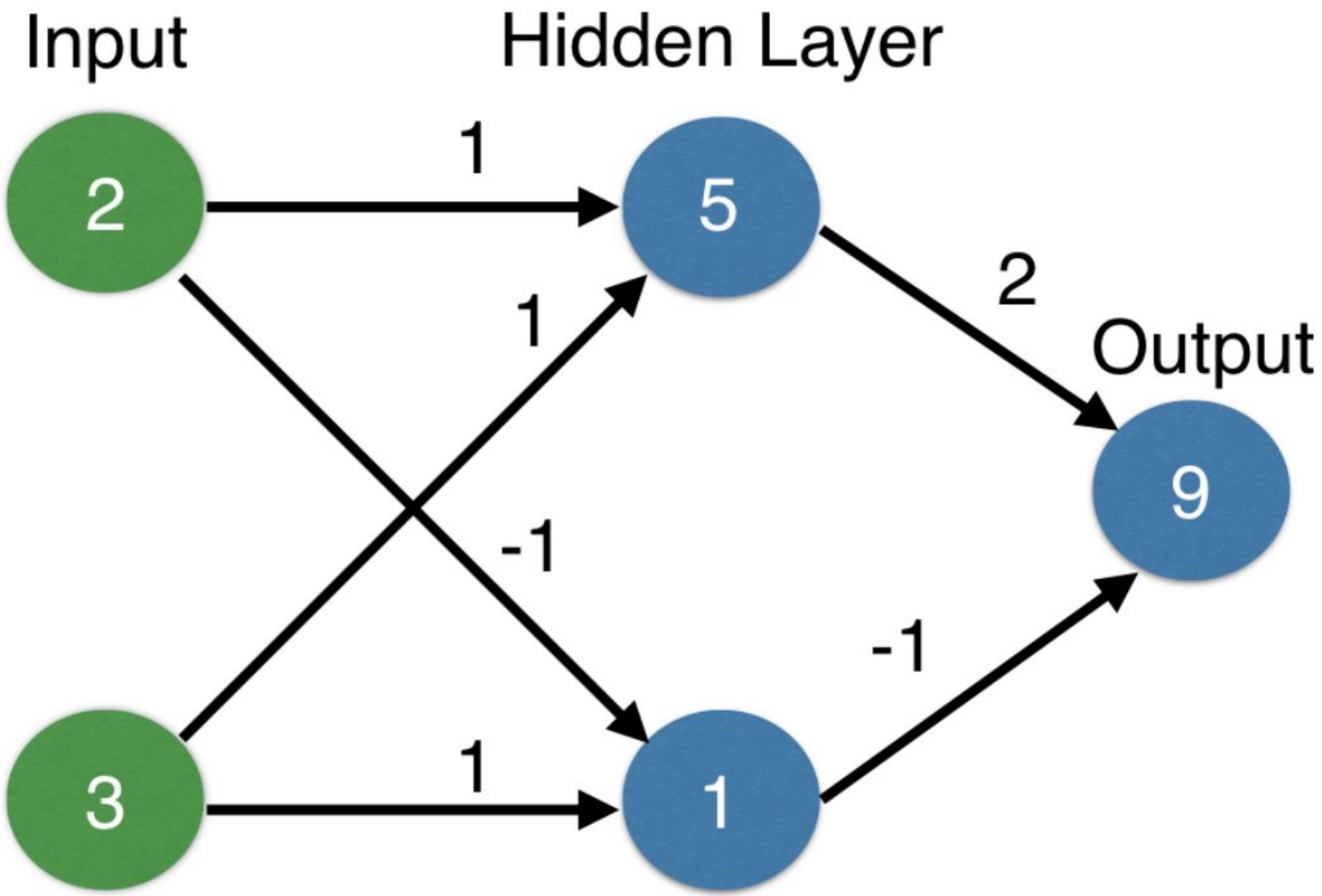
Forward propagation



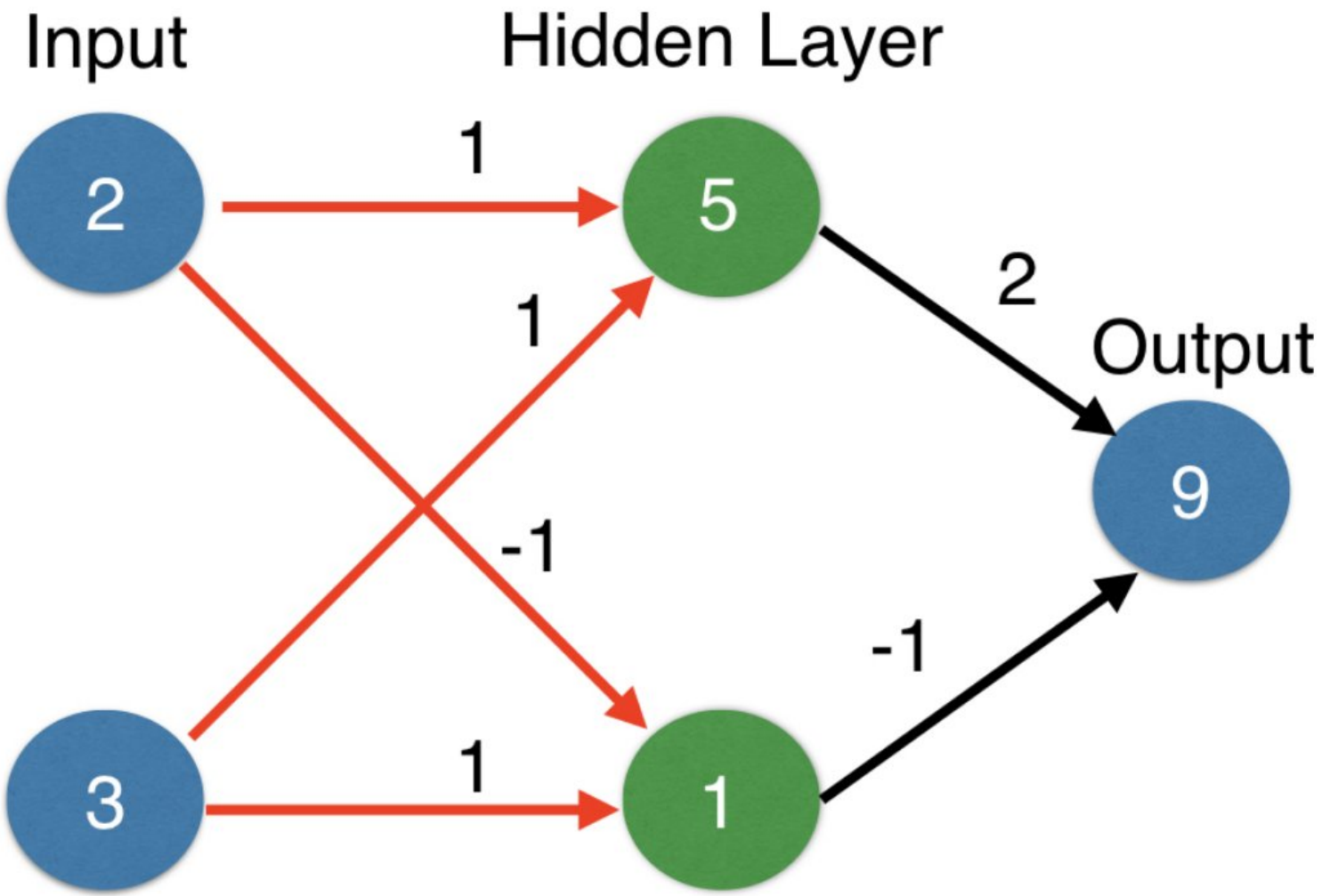
Forward propagation



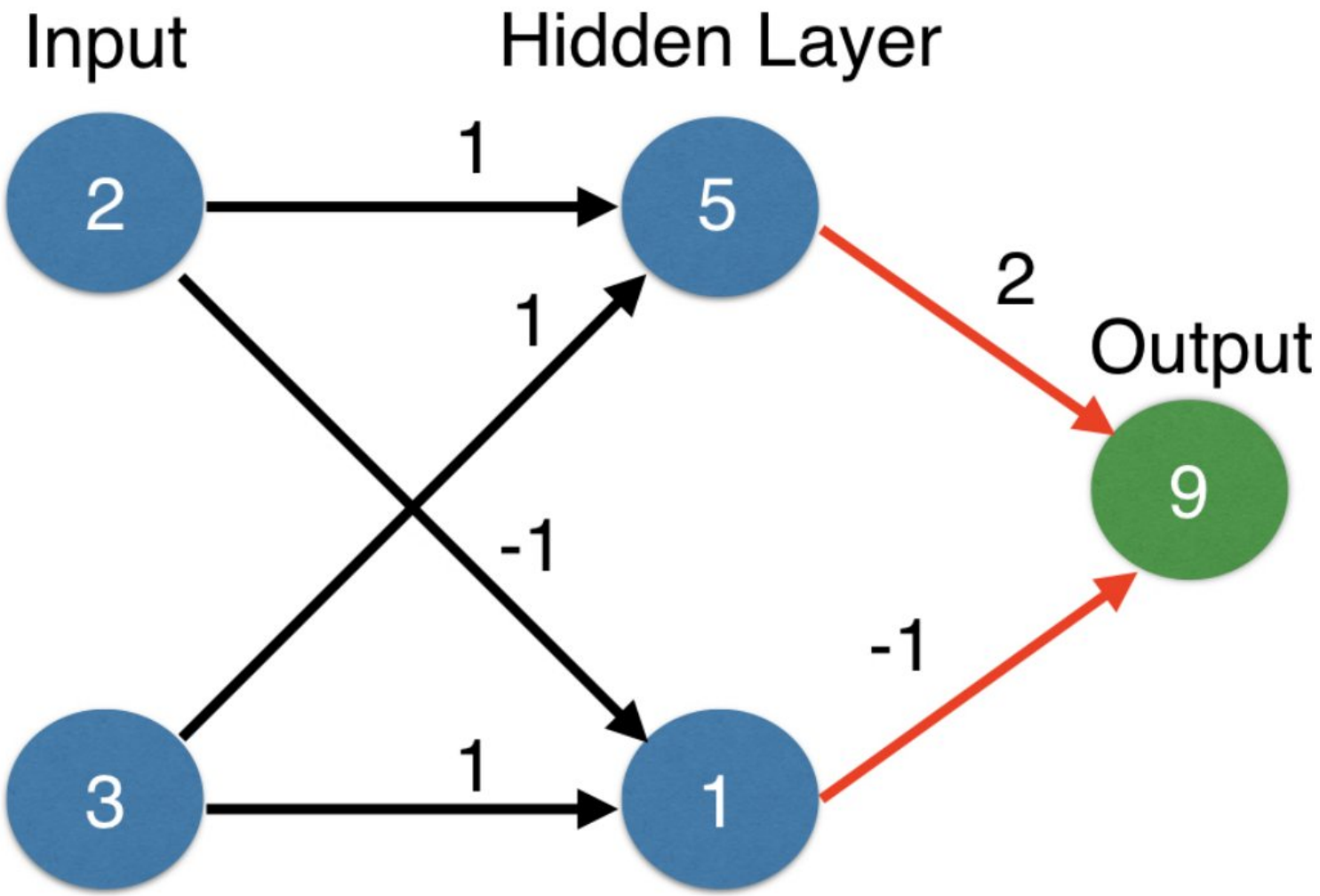
Forward propagation



Forward propagation



Forward propagation



Forward propagation

- Multiply - add process
- Dot product
- Forward propagation for one data point at a time
- Output is the prediction for that data point

Forward propagation code

```
import numpy as np
input_data = np.array([2, 3])
weights = {'node_0': np.array([1, 1]),
           'node_1': np.array([-1, 1]),
           'output': np.array([2, -1])}
node_0_value = (input_data * weights['node_0']).sum()
node_1_value = (input_data * weights['node_1']).sum()
```

Forward propagation code

```
hidden_layer_values = np.array([node_0_value, node_1_value])
print(hidden_layer_values)
```

```
[5, 1]
```

```
output = (hidden_layer_values * weights['output']).sum()
print(output)
```

```
9
```

Let's practice!

INTRODUCTION TO DEEP LEARNING IN PYTHON

Activation functions

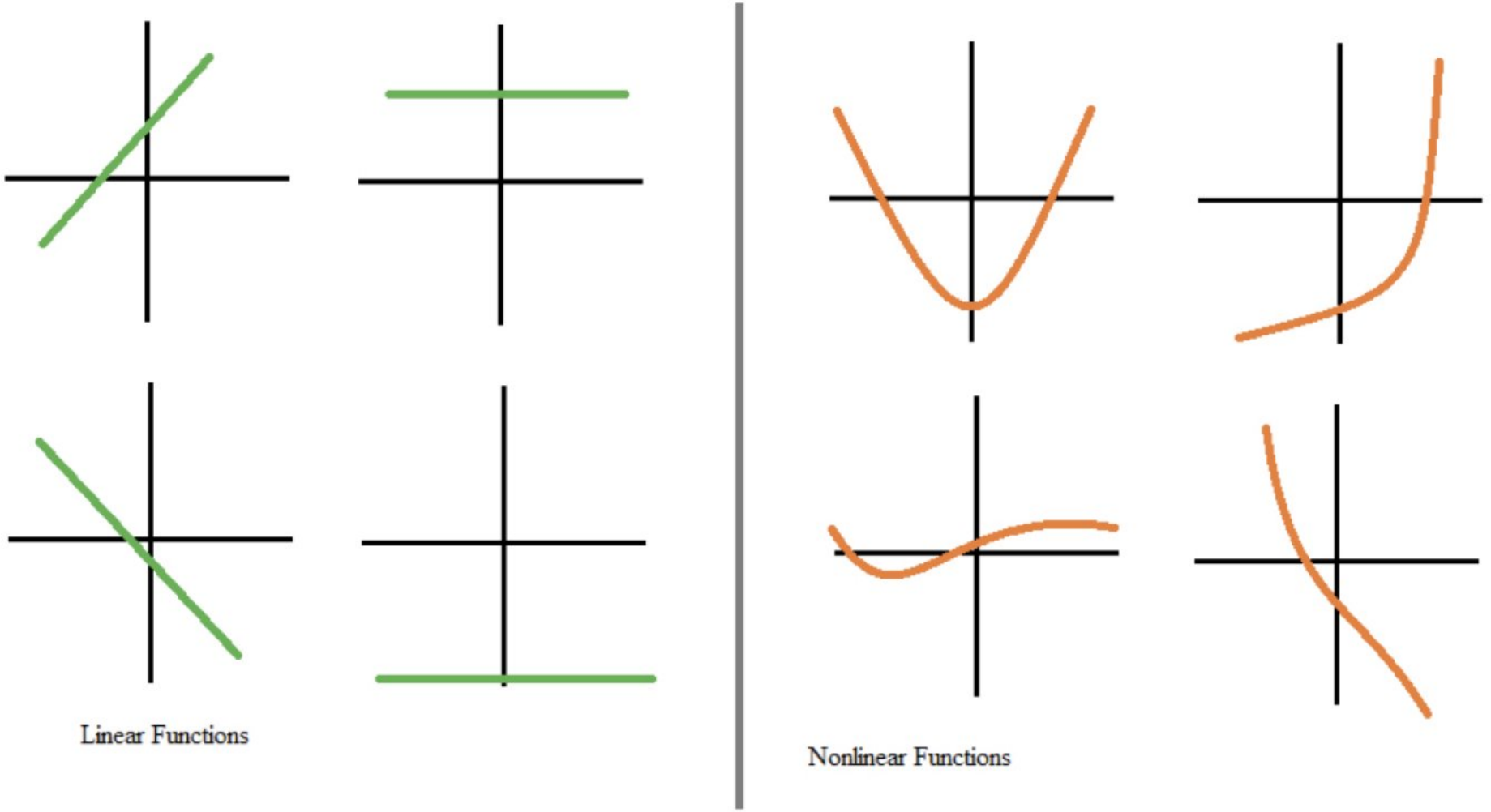
INTRODUCTION TO DEEP LEARNING IN PYTHON



Dan Becker

Data Scientist and contributor to Keras and TensorFlow libraries

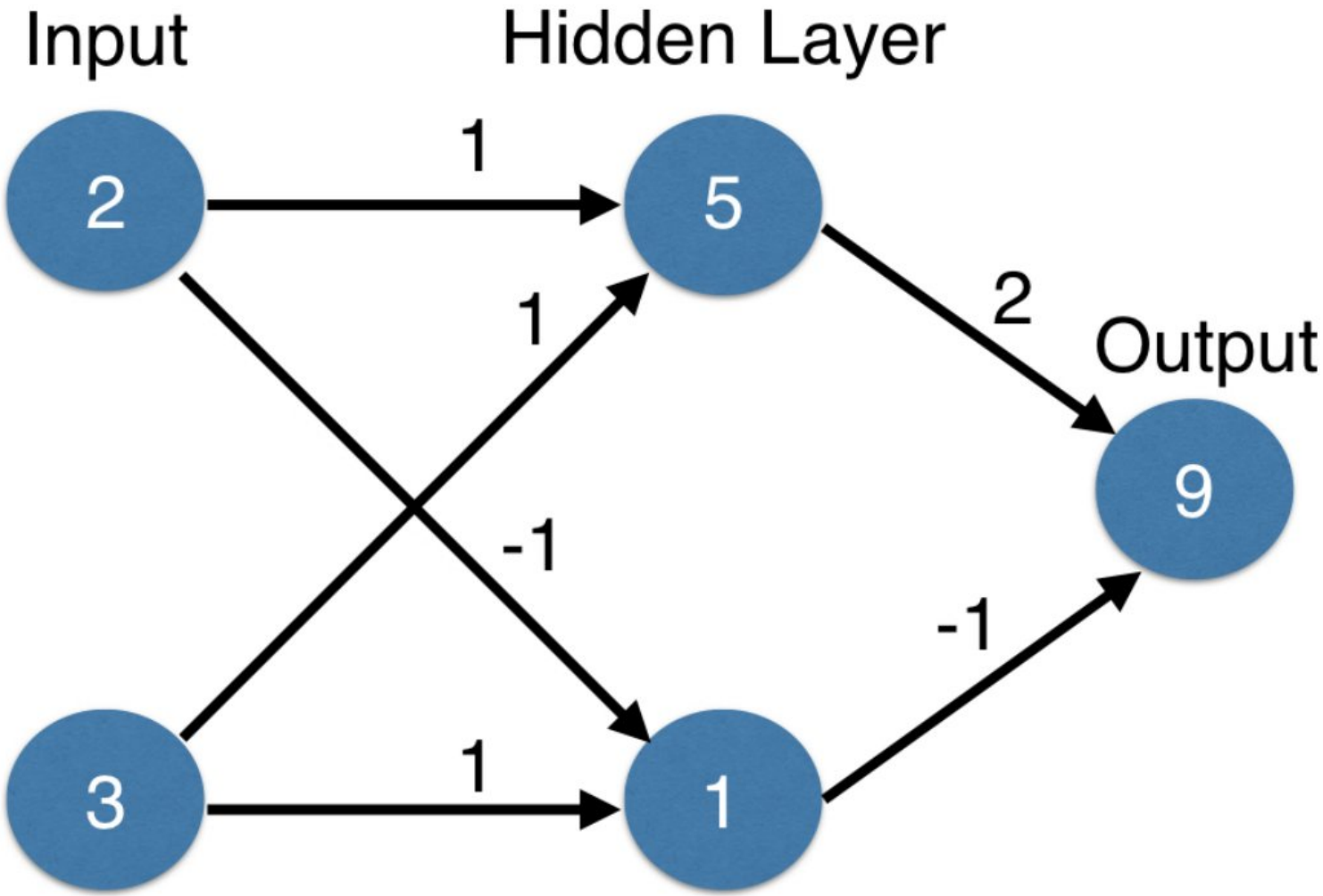
Linear vs. non-linear Functions



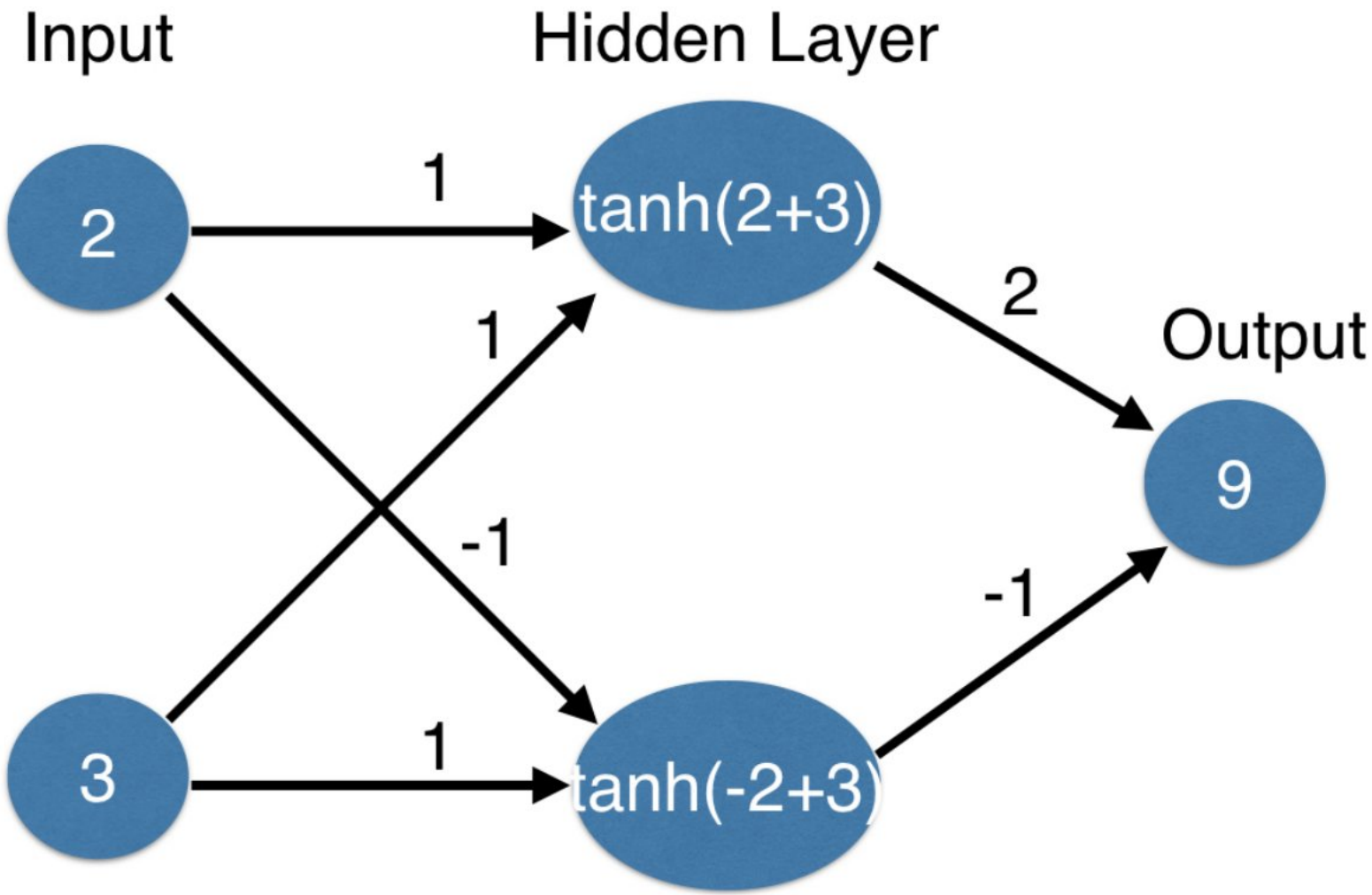
Activation functions

- Applied to node inputs to produce node output

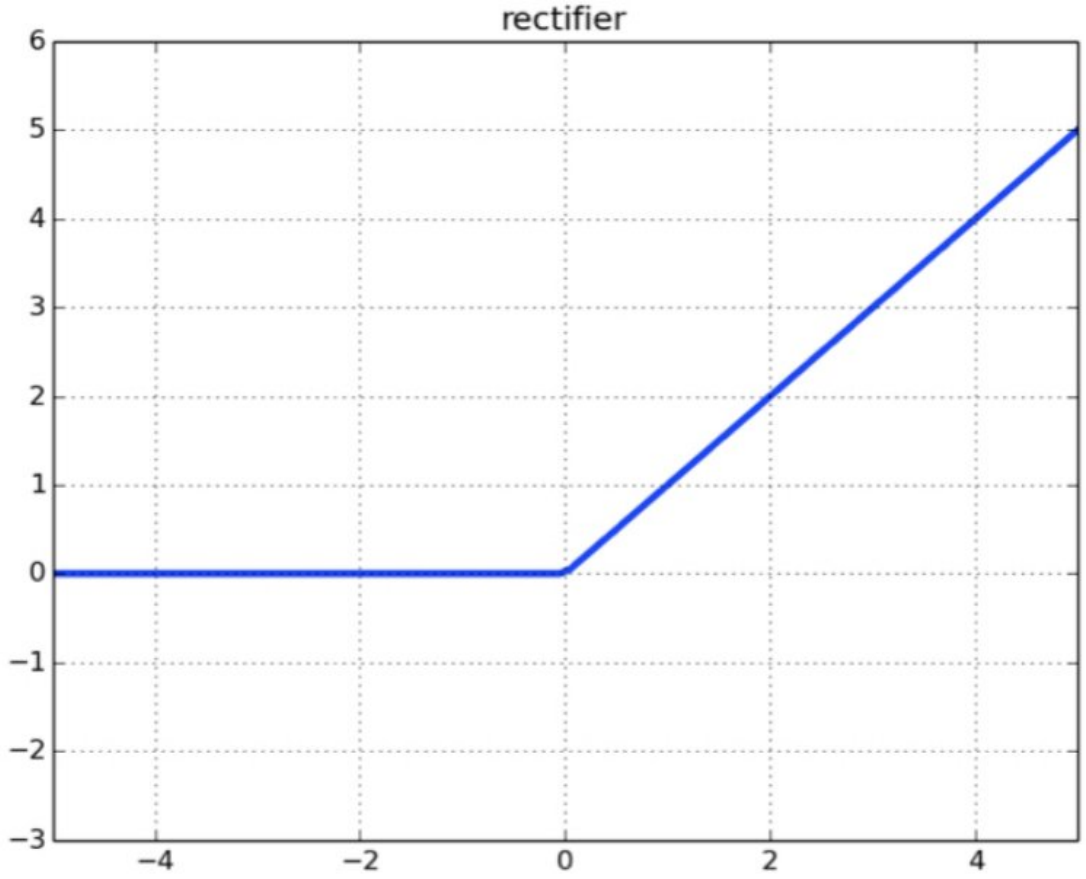
Improving our neural network



Activation functions



ReLU (Rectified Linear Activation)



$$\text{RELU}(x) = \begin{cases} 0 \text{ if } x < 0 \\ x \text{ if } x >= 0 \end{cases}$$

Activation functions

```
import numpy as np
input_data = np.array([-1, 2])
weights = {'node_0': np.array([3, 3]),
           'node_1': np.array([1, 5]),
           'output': np.array([2, -1])}
node_0_input = (input_data * weights['node_0']).sum()
node_0_output = np.tanh(node_0_input)
node_1_input = (input_data * weights['node_1']).sum()
node_1_output = np.tanh(node_1_input)
hidden_layer_outputs = np.array([node_0_output, node_1_output])
output = (hidden_layer_outputs * weights['output']).sum()
```

```
print(output)
```

```
1.2382242525694254
```

Let's practice!

INTRODUCTION TO DEEP LEARNING IN PYTHON

Deeper networks

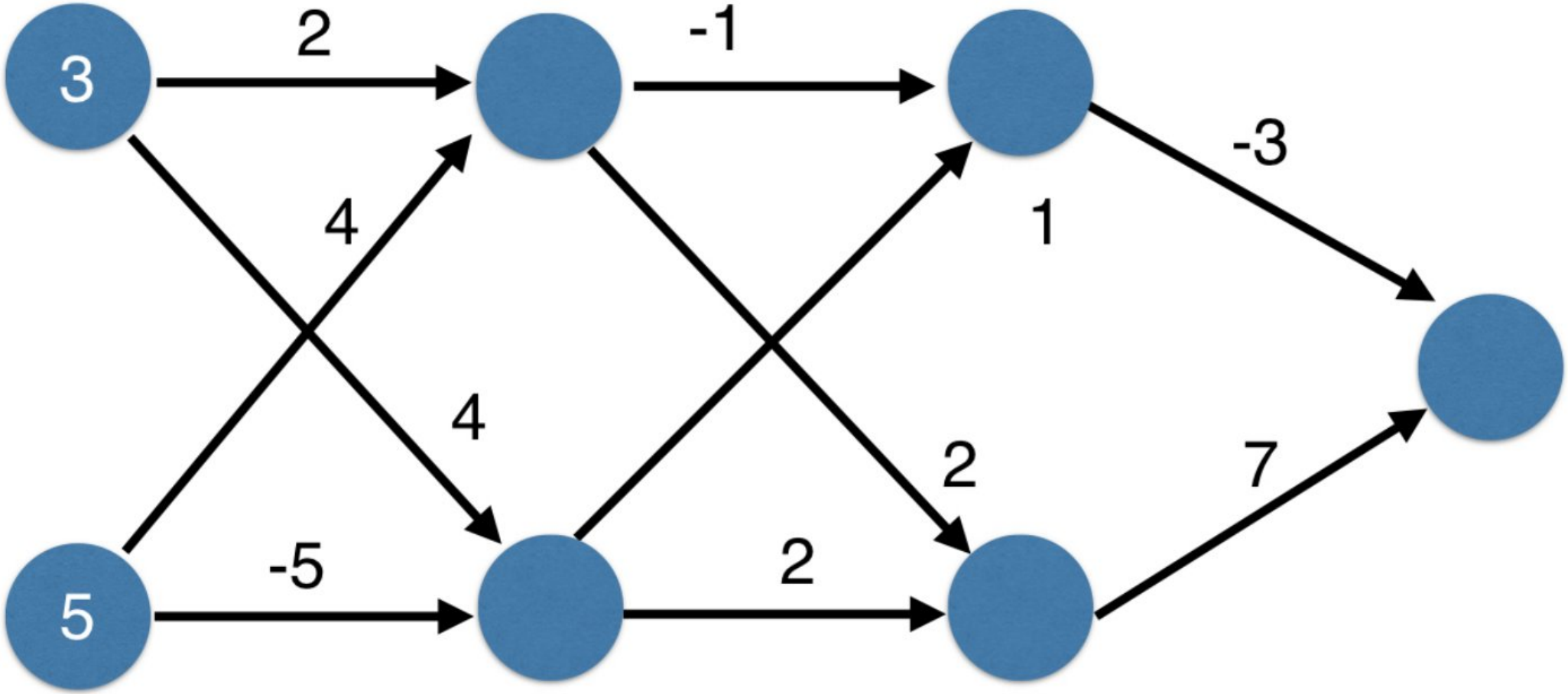
INTRODUCTION TO DEEP LEARNING IN PYTHON



Dan Becker

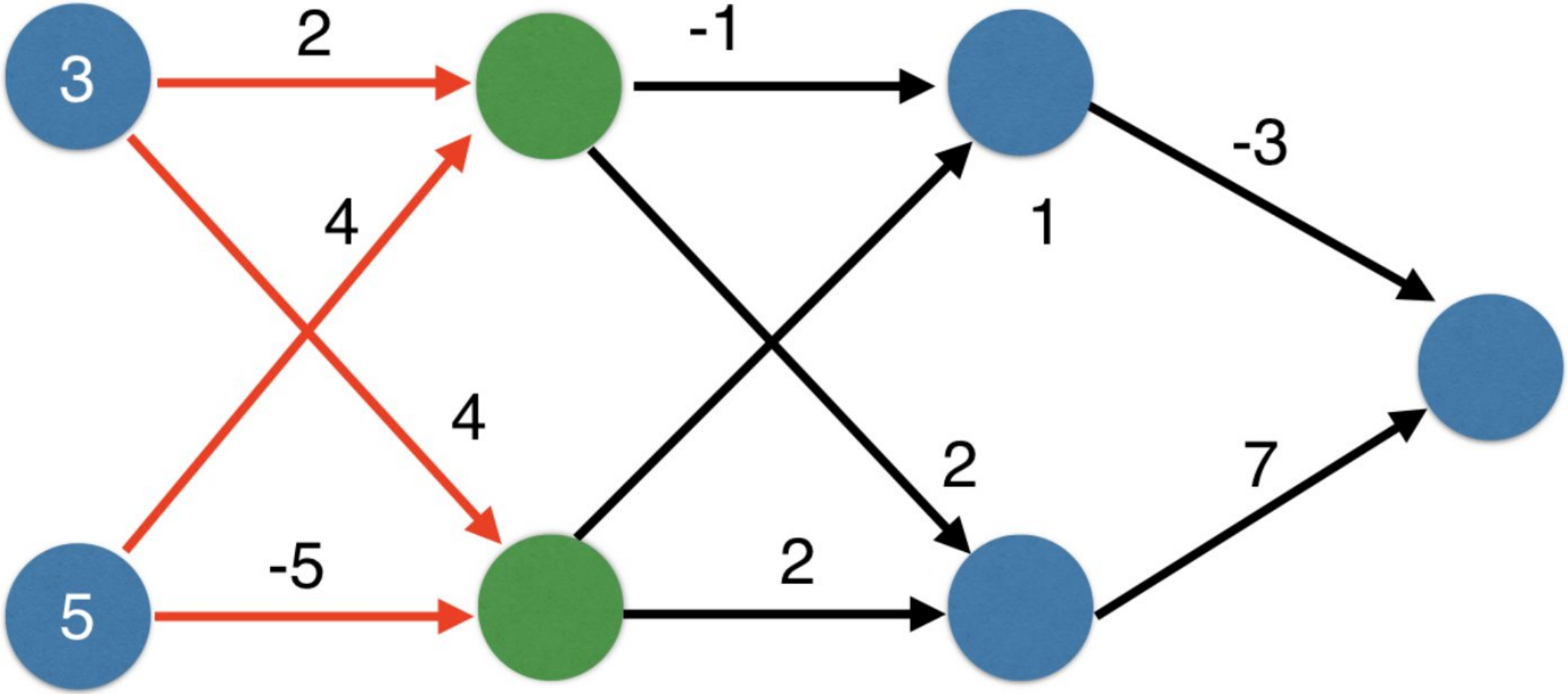
Data Scientist and contributor to Keras and TensorFlow libraries

Multiple hidden layers



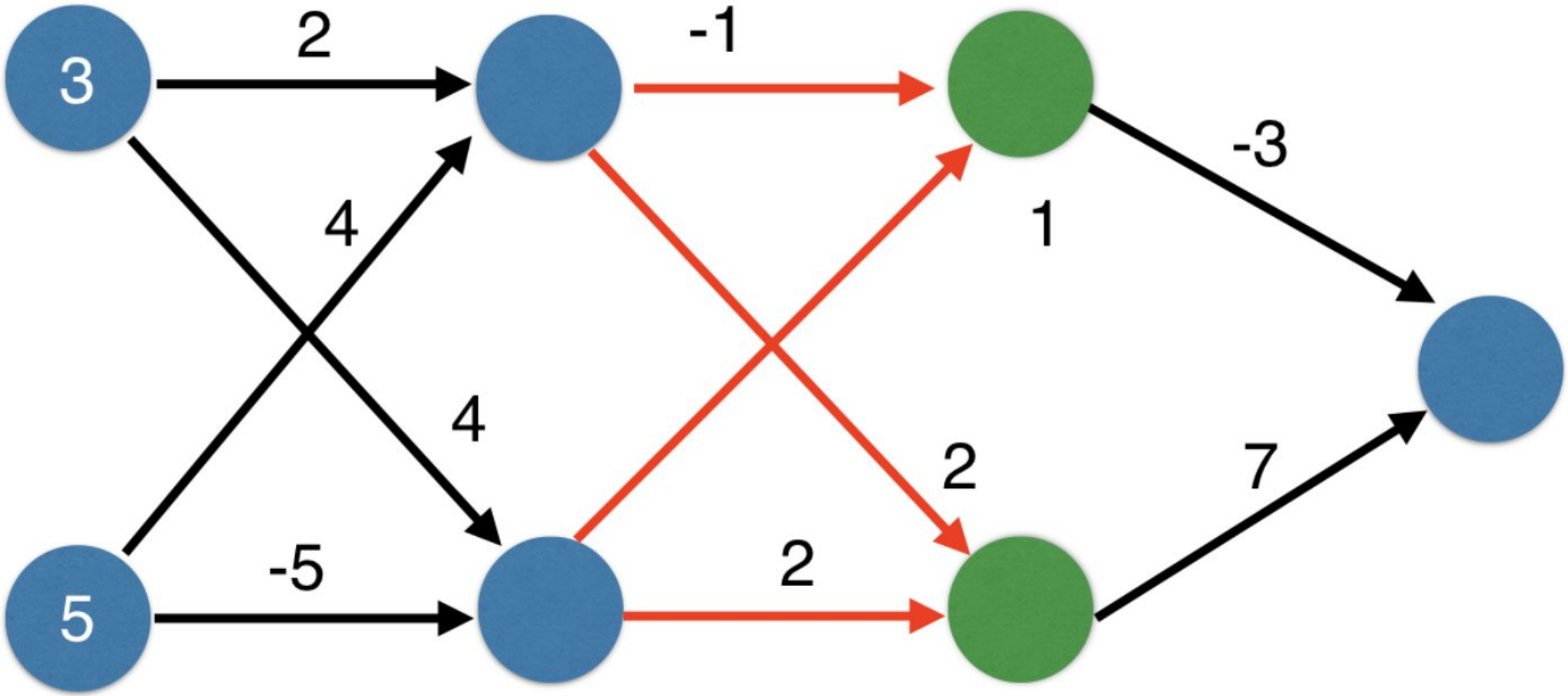
Calculate with ReLU Activation Function

Multiple hidden layers



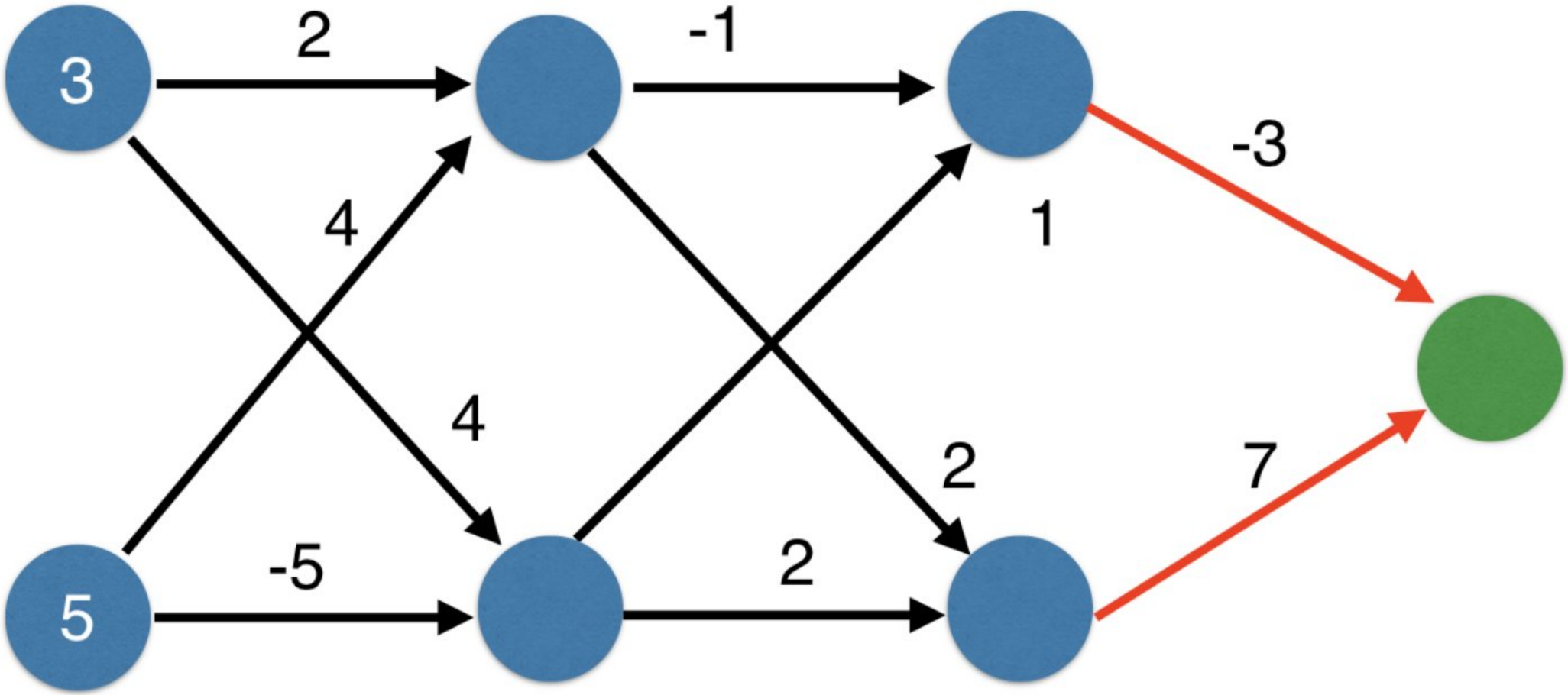
Calculate with ReLU Activation Function

Multiple hidden layers



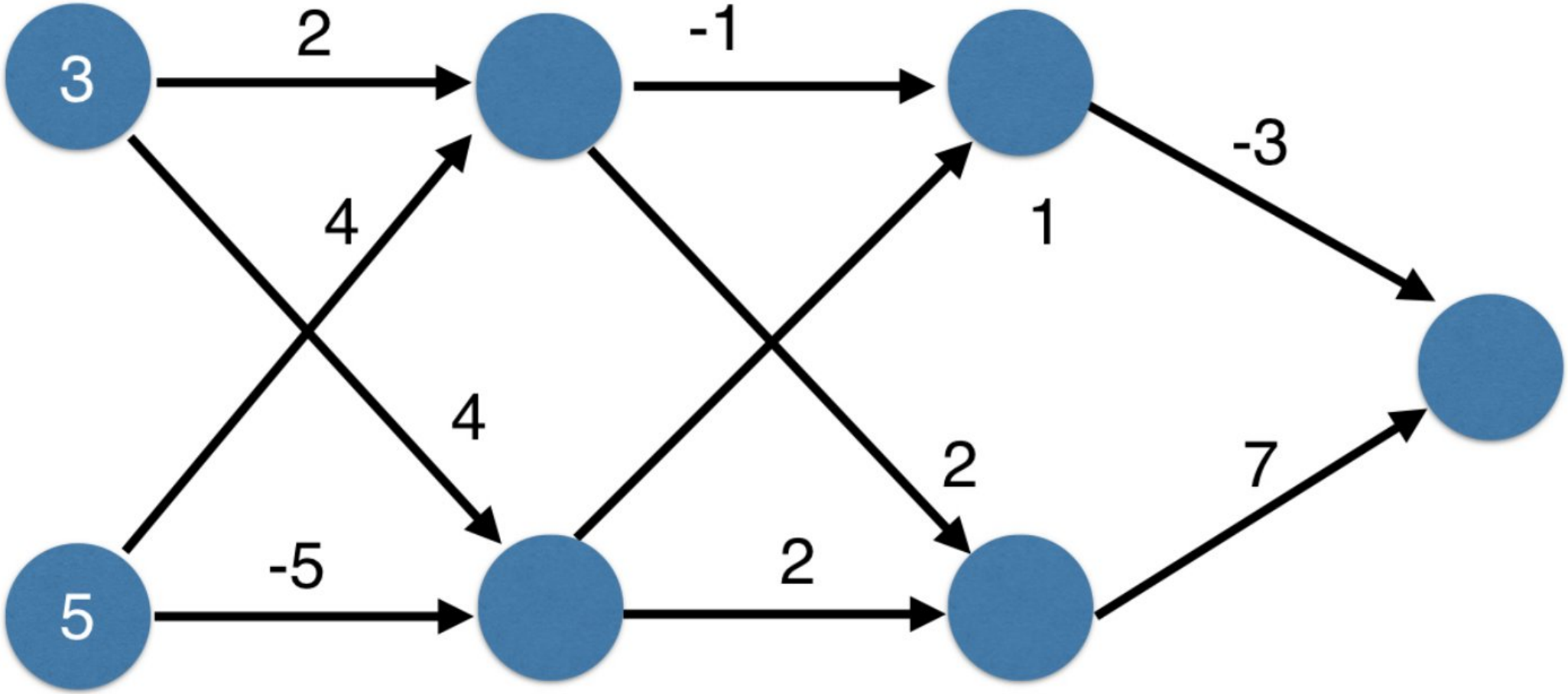
Calculate with ReLU Activation Function

Multiple hidden layers



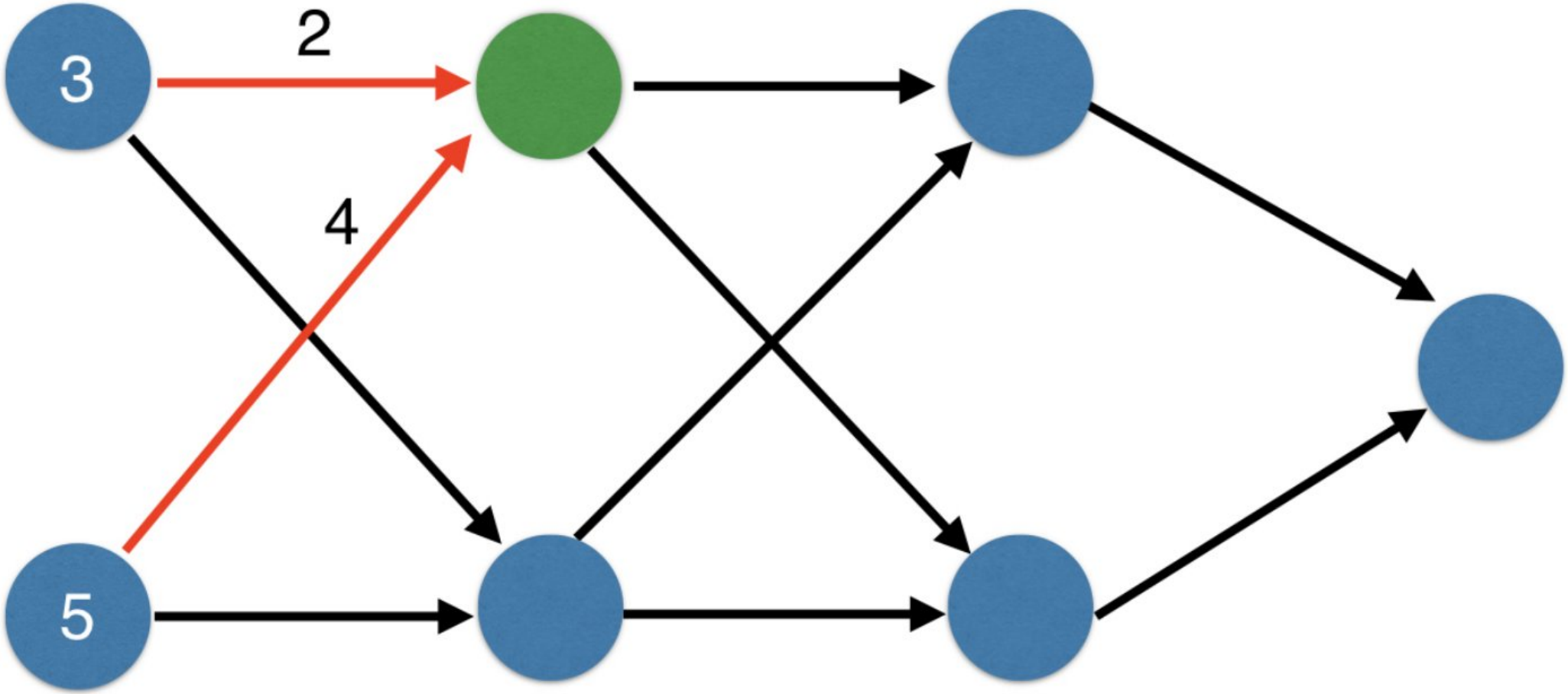
Calculate with ReLU Activation Function

Multiple hidden layers



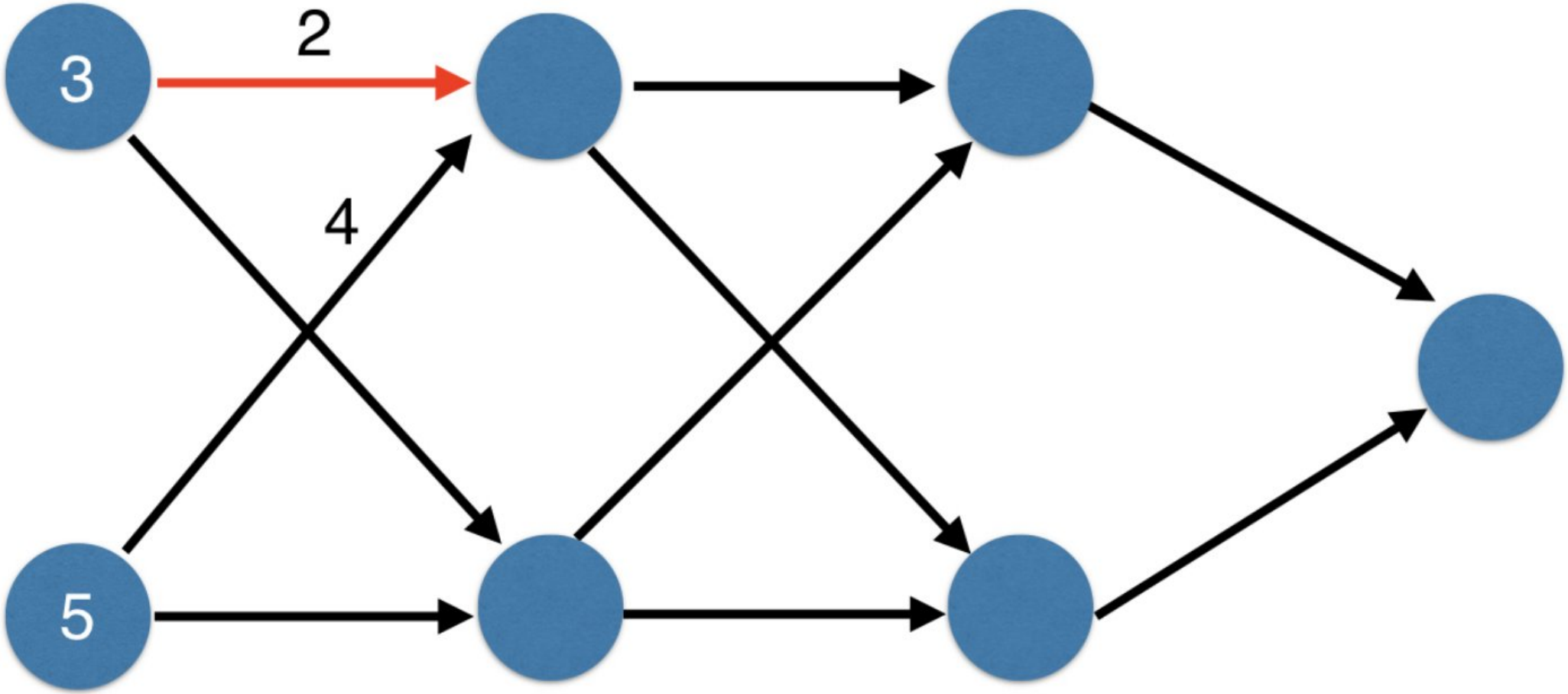
Calculate with ReLU Activation Function

Multiple hidden layers



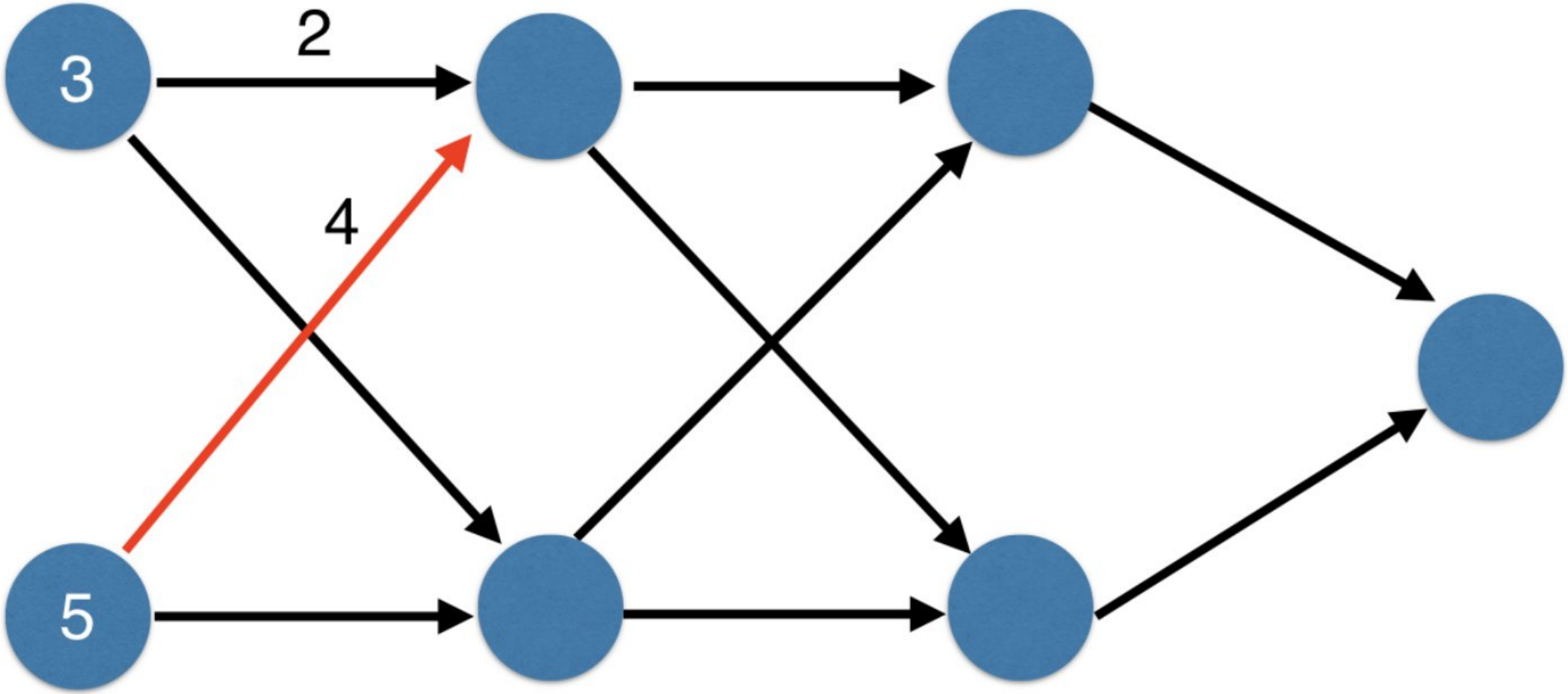
Calculate with ReLU Activation Function

Multiple hidden layers



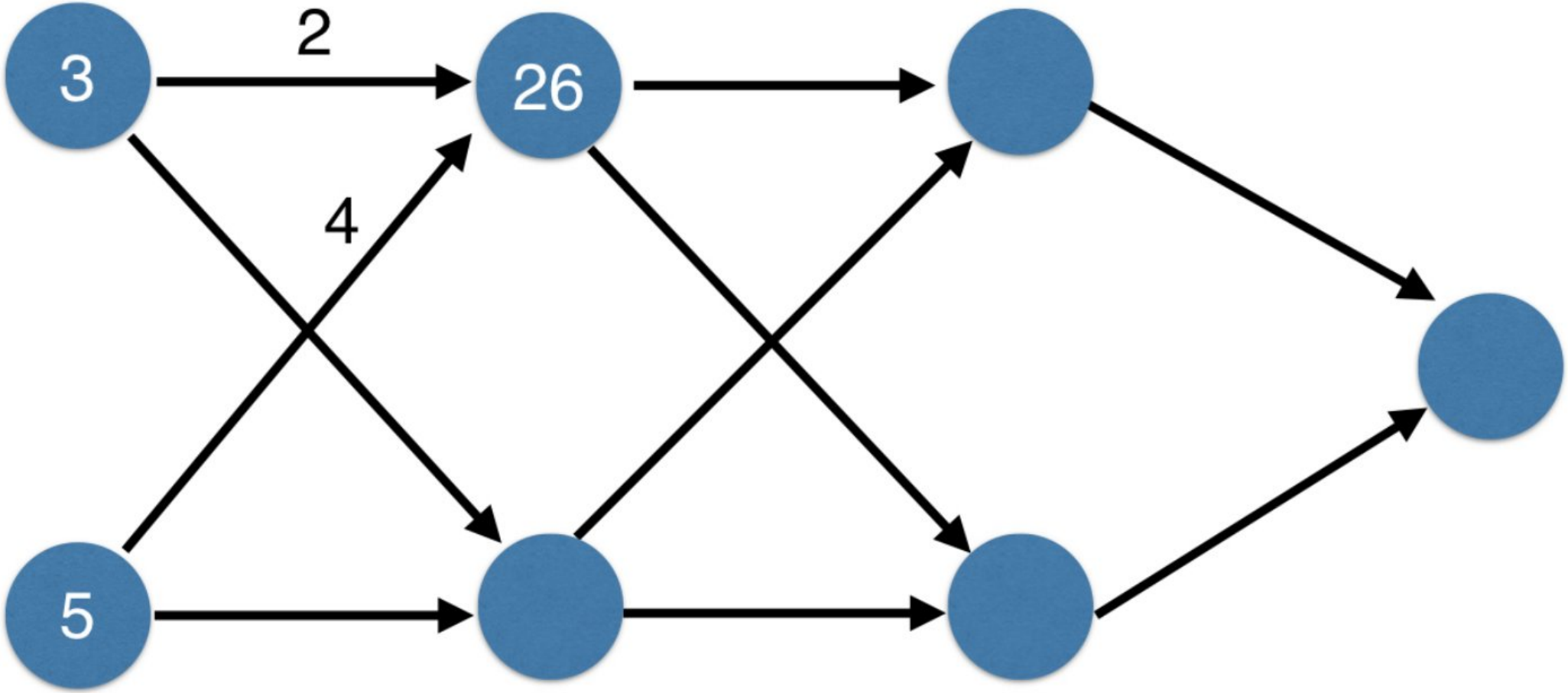
Calculate with ReLU Activation Function

Multiple hidden layers



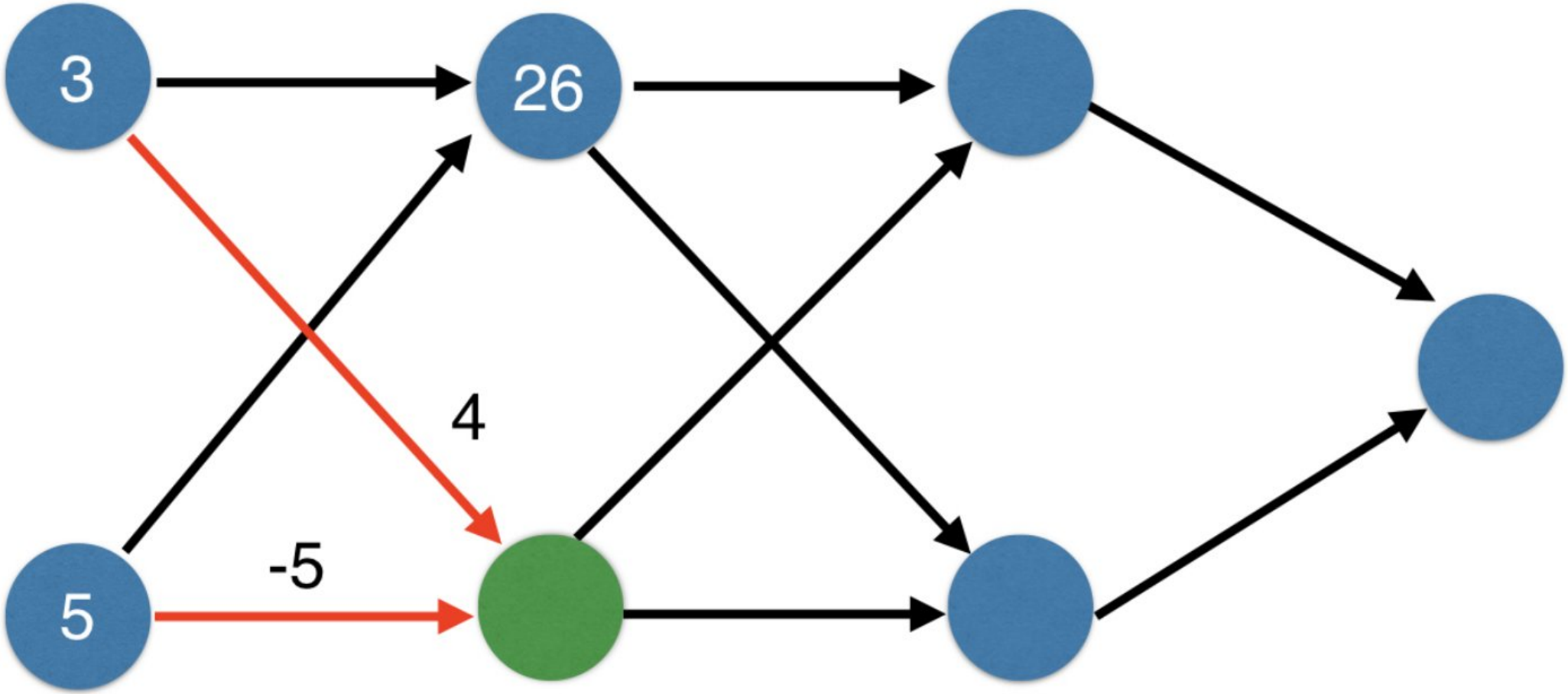
Calculate with ReLU Activation Function

Multiple hidden layers



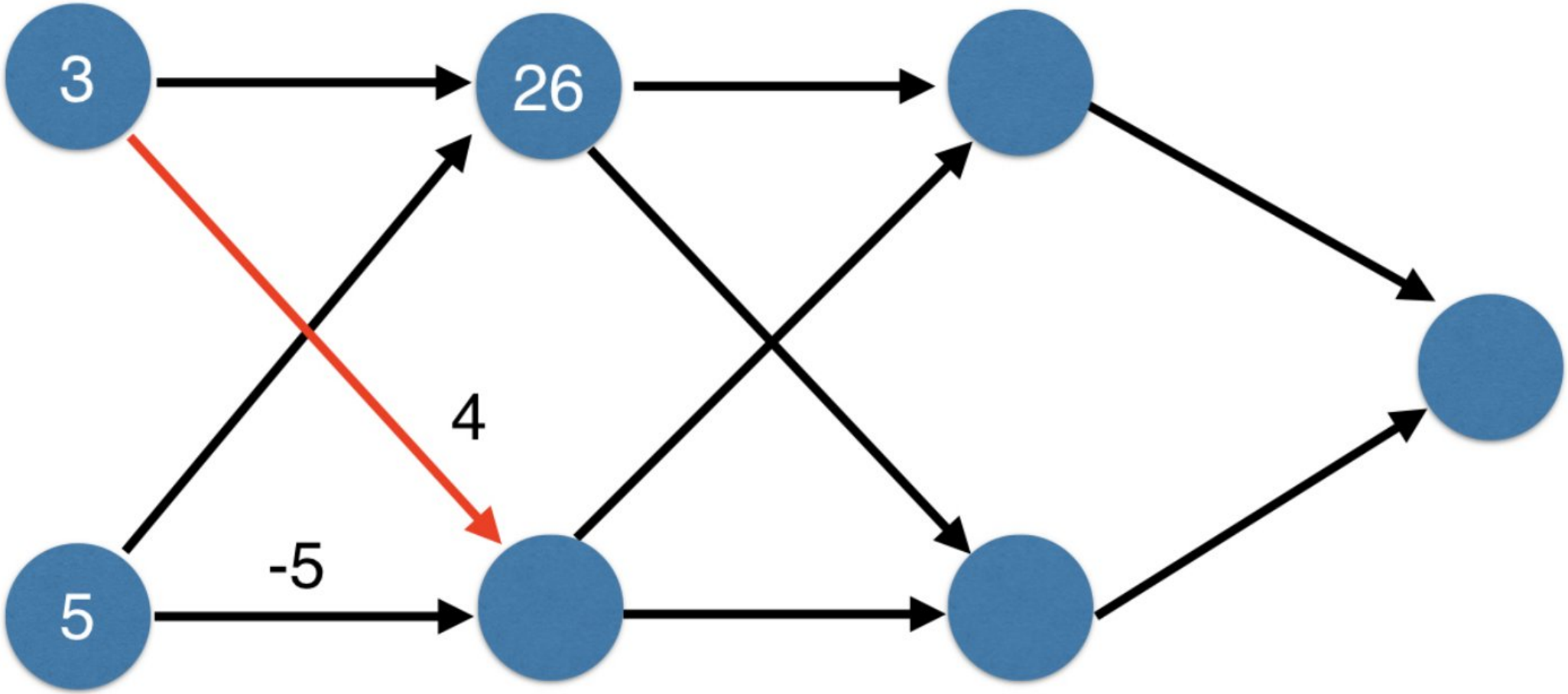
Calculate with ReLU Activation Function

Multiple hidden layers



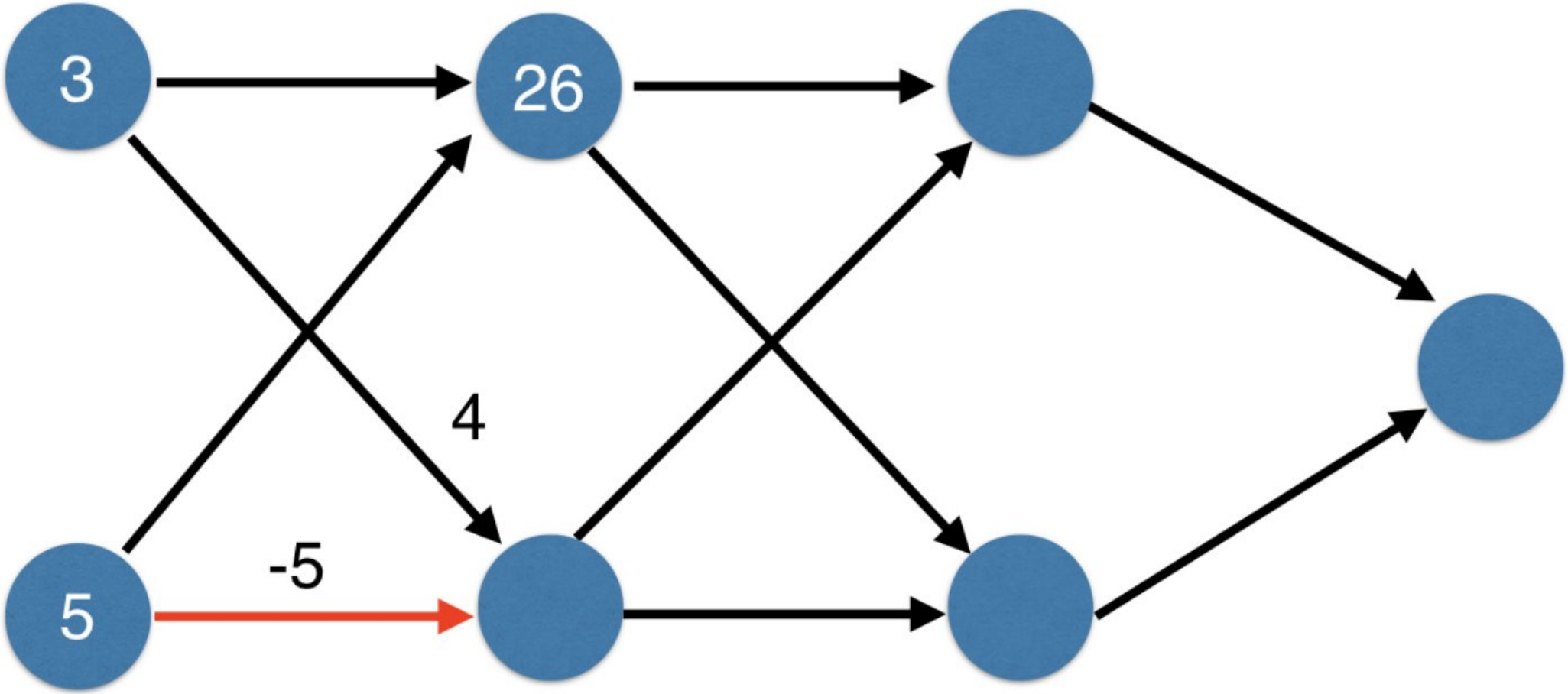
Calculate with ReLU Activation Function

Multiple hidden layers



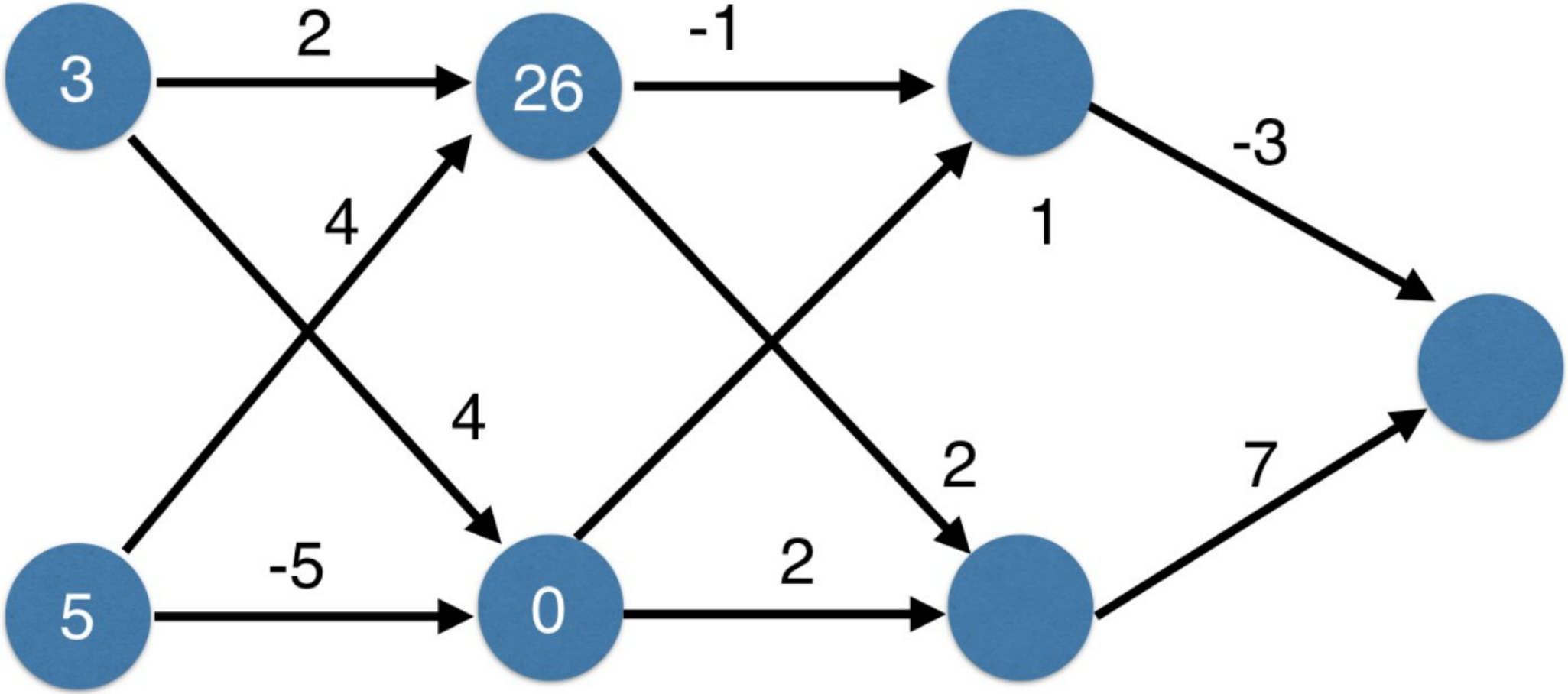
Calculate with ReLU Activation Function

Multiple hidden layers



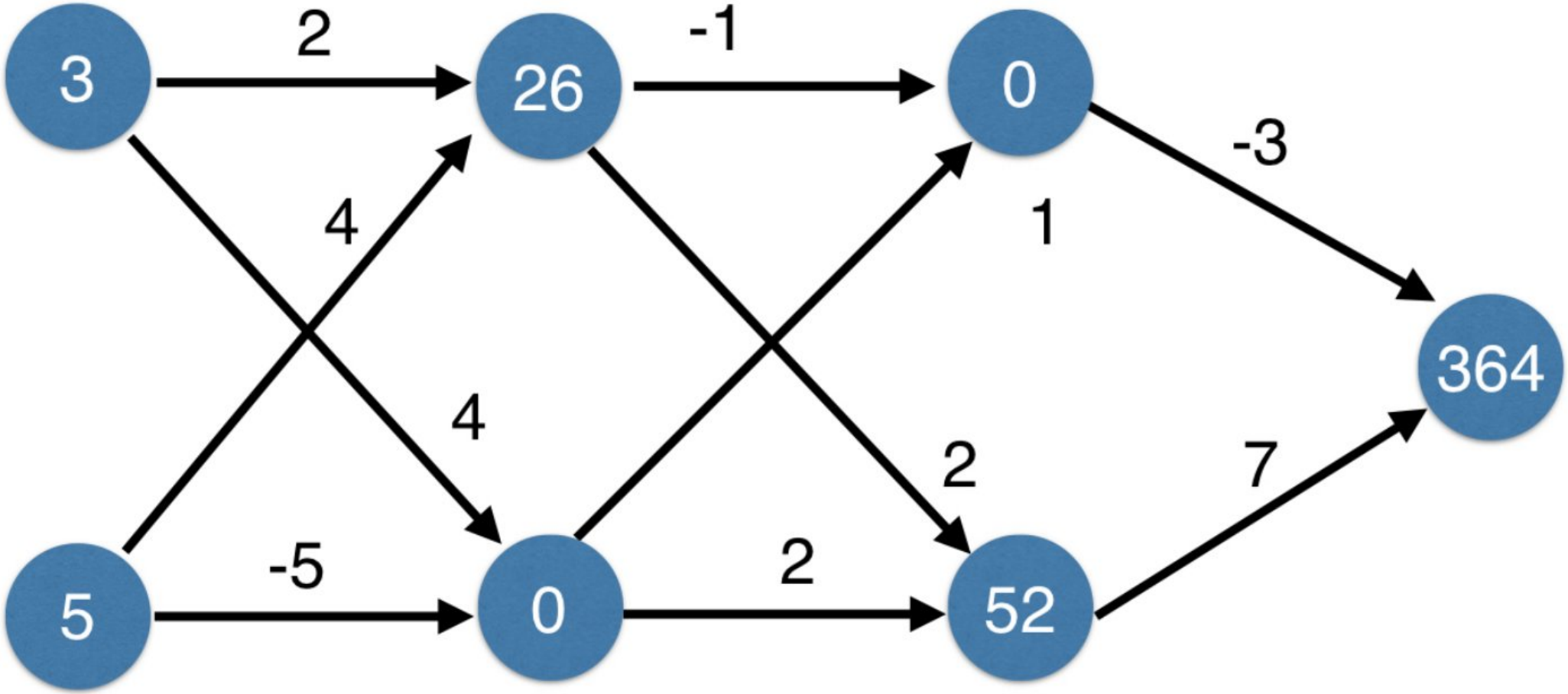
Calculate with ReLU Activation Function

Multiple hidden layers



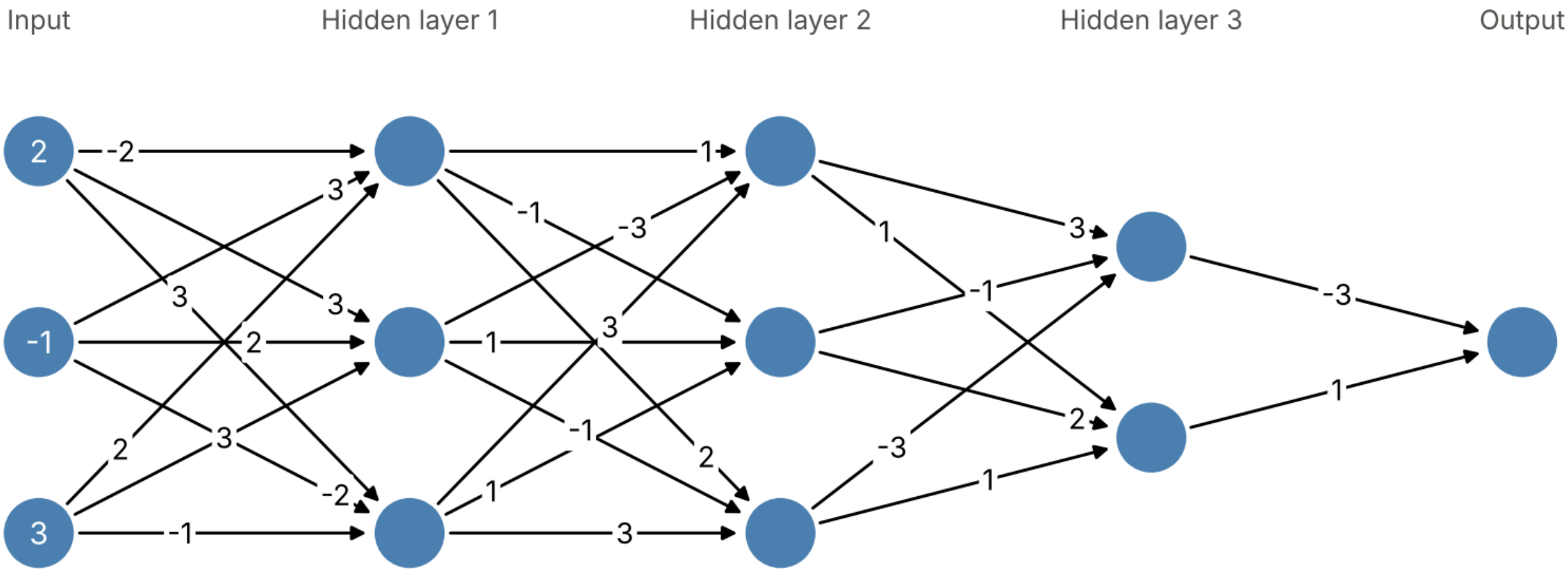
Calculate with ReLU Activation Function

Multiple hidden layers



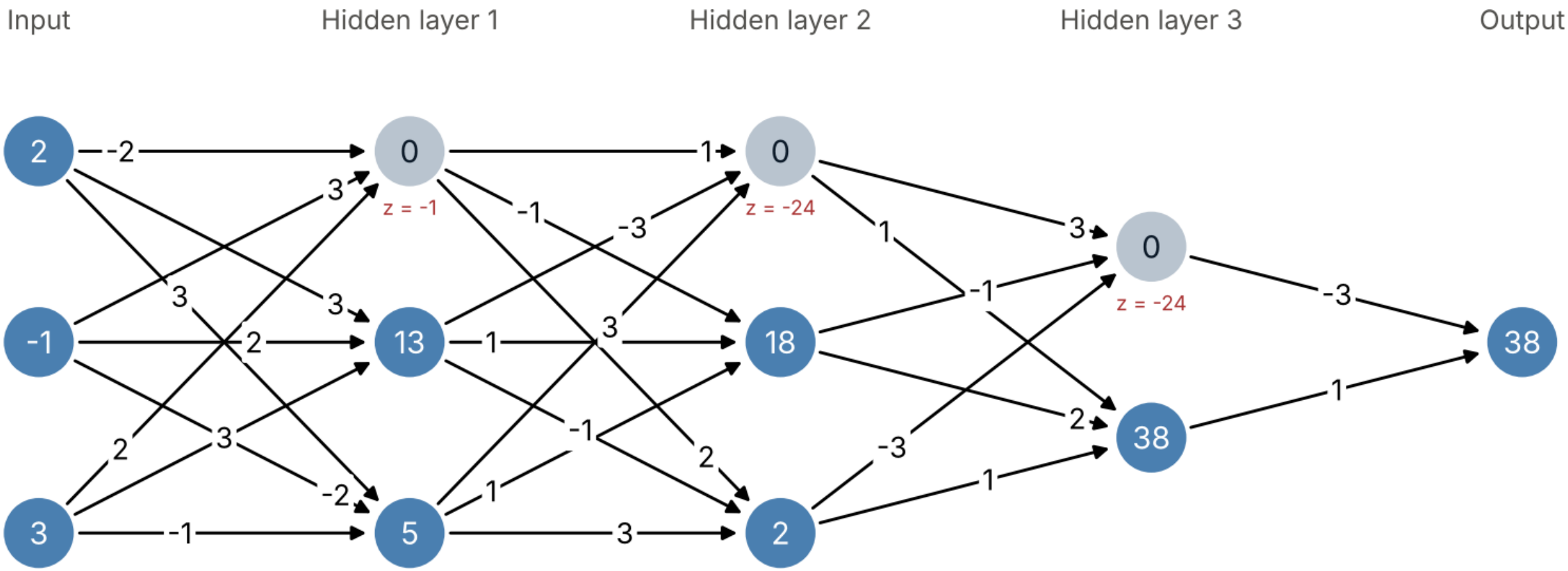
Calculate with ReLU Activation Function

Multiple hidden layers



Calculate with ReLU Activation Function

Multiple hidden layers



Calculate with ReLU Activation Function (grey node: input was negative, ReLU clips it to 0)